REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD POLITECNICA TERRITORIAL DE ARAGUA
"FEDERICO BRITO FIGUEROA"
LA VICTORIA ESTADO ARAGUA
COORDINACION DE CREACION INTELECTUAL Y DESARROLLO SOCIOPRODUCTIVO

Autores:
David Obregon
Diosey Lugo
Tutor:
Prof. Carlos Aponte
Fecha: abril de 2026

LÍNEA DE INVESTIGACIÓN: ELECTROMEDICINA

DESARROLLO DE UN ELECTROENCEFALÓGRAFO PROTOTIPO PARA LA ADQUISICIÓN Y REGISTRO DE LA ACTIVIDAD ELÉCTRICA DEL CEREBRO

RESUMEN

Esta propuesta de proyecto de grado plantea el desarrollo de un prototipo de electroencefalógrafo el cual pueda captar y registrar las señales eléctricas del cerebro, siendo capaz de graficar las señales en el dominio del tiempo y en el dominio de la frecuencia, con la opción de aplicar filtrado digital de topologia FIR de 31 saltos con ventana de Hamming y de topologia IIR de 1er orden, ambos con configuraciones pasa-bajas y pasa-altas. Su realización se llevará a cabo mediante las distintas herramientas

tecnológicas que permitan diseñar un circuito de adquisición de datos adecuado para la actividad eléctrica del cerebro y un firmware para un microcontrolador capaz de proveer una comunicación inalámbrica con cualquier dispositivo inteligente que tenga acceso a un navegador web, de modo que puedan visualizar las señales por este medio. Esta propuesta tiene el fin de sentar una base sólida para el desarrollo de futuros equipos en el área de electromedicina, contribuyendo de esta manera a la diversificación de las áreas de conocimiento de los futuros profesionales en el área de electrónica y llegando a ser una alternativa para los centros de salud que necesiten de este tipo de equipo.

PALABRAS CLAVES: Espectro de potencia, microcontrolador, prototipo, ondas, frecuencia, análisis, programación, C, HTML, cerebro, inalámbrico, ESP32.

DISEÑO Y CONSTRUCCIÓN DEL PROTOTIPO

El desarrollo del prototipo de EEG se llevó a cabo considerando la magnitud de las señales eléctricas cerebrales y el ruido que estas podrían tener; por ello, se diseñó un circuito de adquisición de datos que amplifica, filtra y adapta las señales de manera que éstas se puedan procesar digitalmente para su posterior visualización, dándole paso al cumplimiento de los objetivos trazados por los investigadores.

En concordancia con lo antes expuesto, cabe destacar ante todo, que este prototipo de EEG capta las señales mediante electrodos superficiales de copa de oro en una configuración bipolar, lo que significa que se mide la diferencia entre dos zonas eléctricamente activas del cráneo; a su vez, se debe señalar que un par de electrodos diferenciales es considerado una derivación e independientemente del número de derivaciones, debe existir un electrodo de referencia con un potencial tan cercano a cero como sea posible; de manera pues, que se obtengan medidas precisas.

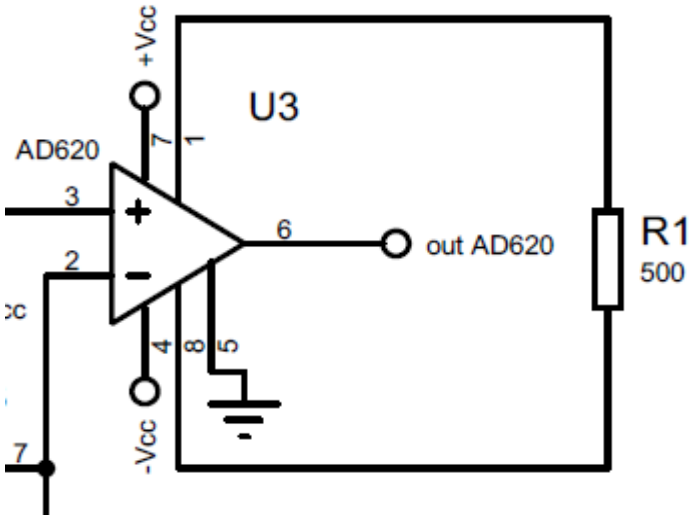
Dentro de este contexto, para una mejor interpretación se procede a detallar cada etapa que conforma al circuito de adquisición de datos: La primera etapa del circuito está compuesta por un amplificador de bio-potenciales, conformado por un amplificador de instrumentación y un amplificador no inversor que aumenta la magnitud de las señales, la segunda etapa consta de un filtro pasa-banda cuyo propósito es atenuar todas las señales eléctricas que no provengan del cráneo, la tercera etapa del circuito es el sumador no inversor unitario cuyo fin es agregar un offset a las señales previamente amplificadas y filtradas, esto no es más que un acondicionamiento de las señales adquiridas, y como última etapa del circuito de adquisición de datos, es el conversor análogo-digital que recibe los datos analógicos y los convierte en datos digitales y los envía al microcontrolador.

Dado que la magnitud de las señales eléctricas cerebrales oscila entre uno y cien microvoltios (1 y 100uV), es necesario amplificar esas señales, como a su vez, es

necesario reducir el ruido eléctrico causado por imperfecciones en los electrodos, el movimiento de los músculos, los ojos, los latidos del corazón e incluso la respiración del sujeto contribuye a ruido que se puede experimentar en el proceso de adquisición de las señales; la necesidad de amplificar y reducir el ruido eléctrico en la señal da pie a que se utilice el amplificador de instrumentación AD620 y el amplificador operacional TL084.

En este contexto, el AD620 es un amplificador de instrumentación de bajo costo y baja potencia, que posee una gran precisión y tiene una ganancia configurable desde uno hasta diez mil (1 hasta 10.000) mediante una sola resistencia; es ideal para sistemas de adquisición de datos e instrumentación medica debido a su bajo ruido, potencia disipada, baja corriente de bias y gran razón de rechazo en modo común, lo que asegura un grado de precisión aceptable para esta aplicación; a continuación se puede apreciar el circuito implementado:

Figura 5. Amplificador Instrumentación



Elaborado por: Los Investigadores (2025).

En la figura descrita anteriormente, se aprecia que la resistencia R1 conectada entre los terminales 1 y 8 determina la ganancia según la formula determinada por el fabricante en el datasheet, observe la siguiente formula:

$$G = \frac{49.4k\Omega}{R_G} + 1$$

Donde:

G = ganancia del amplificador.

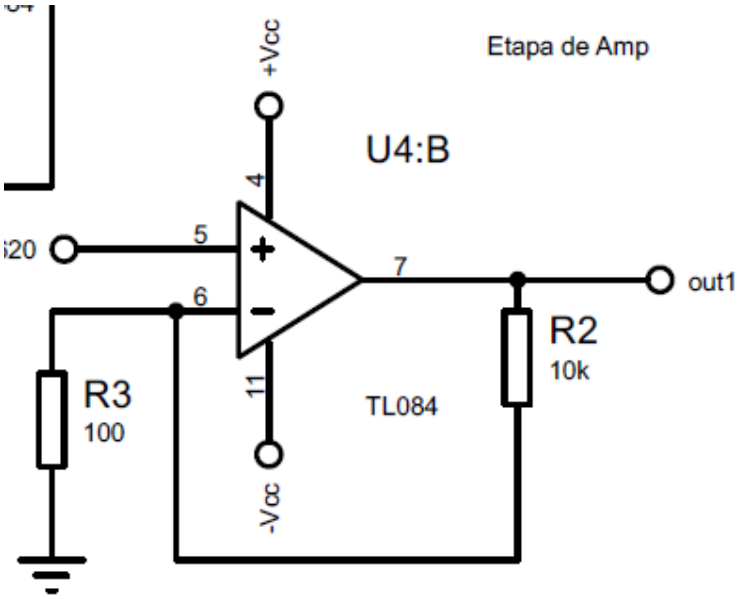
R_G = Resistencia de control de ganancia.

Para este prototipo, considerando que la amplitud de las señales cerebrales es del orden de los micro-voltios, se toma el valor mínimo y el máximo que éstas pueden tener siendo uno y cien microvoltios (1uV y 100uV), respectivamente y dado que es necesario que las señales se encuentren en el orden de los voltios para procesarlas con el conversor analógico-digital, se especifica una ganancia total para el amplificador de diez mil (10.000), dado que:

10.000 x 1uV= 1mV y 10.000 x 100uV = 1V

Es importante señalar que para no saturar al amplificador de instrumentación se dividió esta ganancia entre el amplificador AD620 y un amplificador no inversor de ganancia hecho con el TL084; este es un amplificador operacional de propósito general cuádruple que utiliza tecnología J-FET, posee baja corriente de bias y bajo coeficiente de offset por temperatura y una alta velocidad de respuesta en frecuencia, debido a sus características se determinó que era apropiado para implementar en este prototipo; además, como es cuádruple ayuda a disminuir el número de componentes para el proyecto por lo que es más eficiente, en este caso se configuró como amplificador no inversor como se muestra en la siguiente figura:

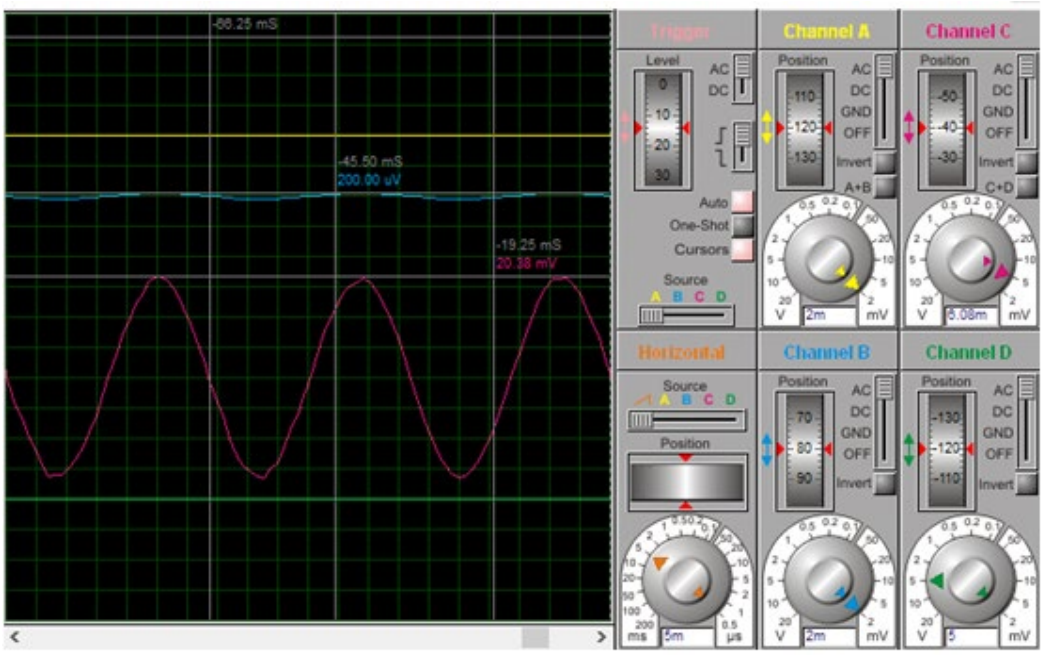
Figura 6. Amplificador No Inversor



Elaborado por: Los Investigadores (2025).

En este sentido, ambos amplificadores están conectados en cascada y cuentan con una ganancia de cien (100) por lo que la ganancia total de la etapa es de diez mil (10.000), lo que es suficiente para esta aplicación. La simulación del circuito realizada comprueba lo anteriormente expuesto y se puede apreciar en la siguiente figura:

Figura 7. Simulación Etapa 1. Amplificador



Elaborado por: Los Investigadores (2025).

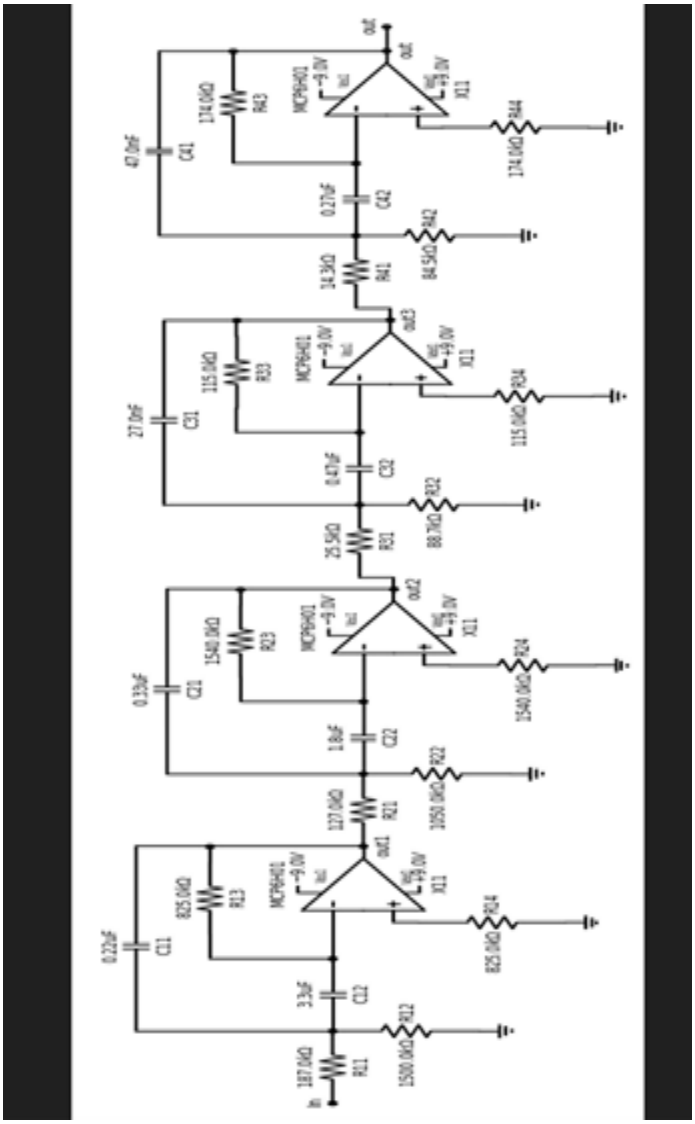
Estas son las señales a la salida del amplificador de instrumentación (Onda Azul) y a la salida del amplificador de ganancia (Onda Roja) para una entrada de 2uV.

Siguiendo esta línea explicativa, la siguiente etapa en desarrollarse fue la del filtrado, la cual está compuesta por un filtro pasa-banda de 8vo orden con topología butterworth cuyas frecuencias de corte son 0.5Hz para la inferior y 30Hz para la superior, esto asegura que sólo se atenúen aquellos armónicos que no pertenezcan a la banda de frecuencia de las señales eléctricas cerebrales.

Para facilitar la implementación del filtro se recurrió al uso de la herramienta FILTERLAB de la empresa MICROCHIP, esta herramienta permite seleccionar el tipo de filtro que se desea calcular, su atenuación en la banda de paso, el orden del filtro e incluso la ganancia del mismo; para este prototipo, se calculó un filtro que satisfaga las

características expuestas de antemano y el resultado se puede apreciar en la siguiente figura:

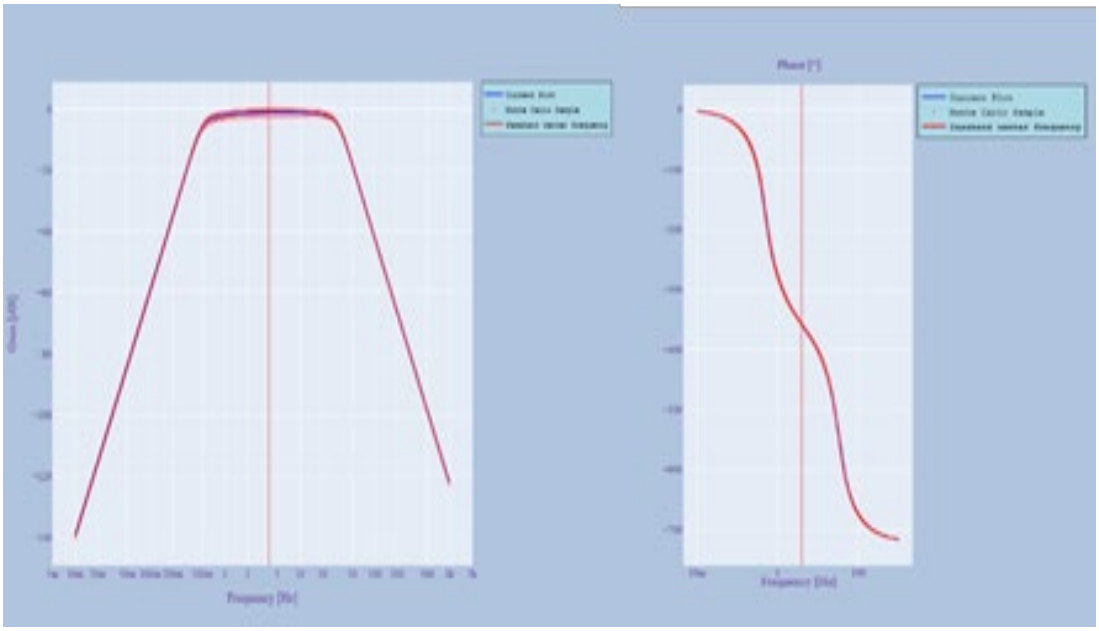
Figura 8. Bosquejo del Filtro



Elaborado por: Los Investigadores (2025).

La respuesta en frecuencia y en fase estimada por FILTERLAB es la siguiente:

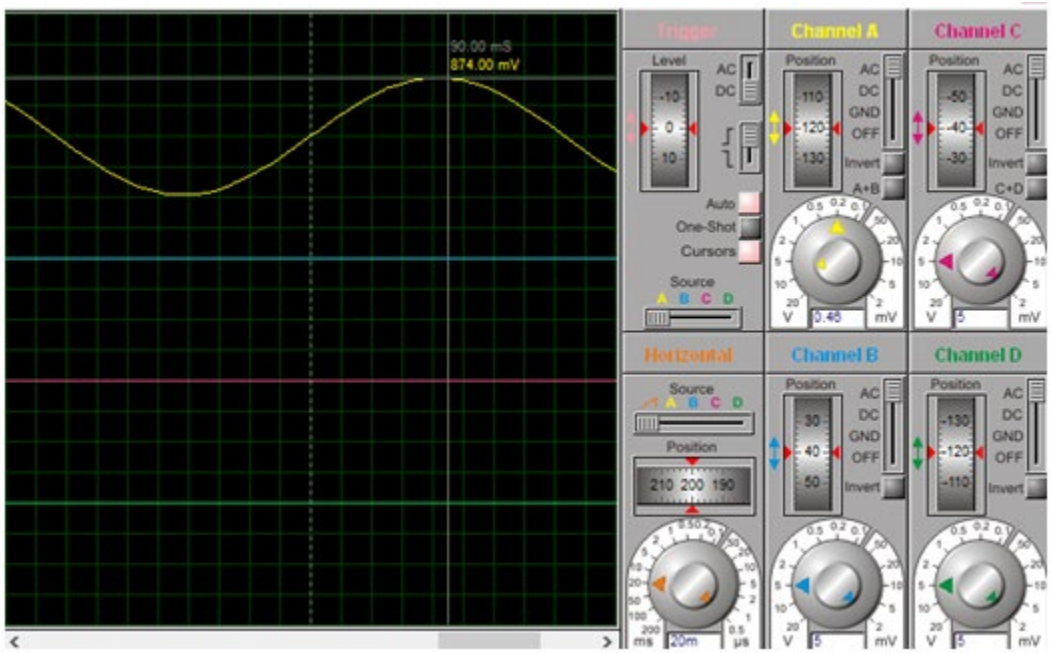
Figura 9. Respuesta en Frecuencia y Fase del Filtro



Elaborado por: Los Investigadores (2025).

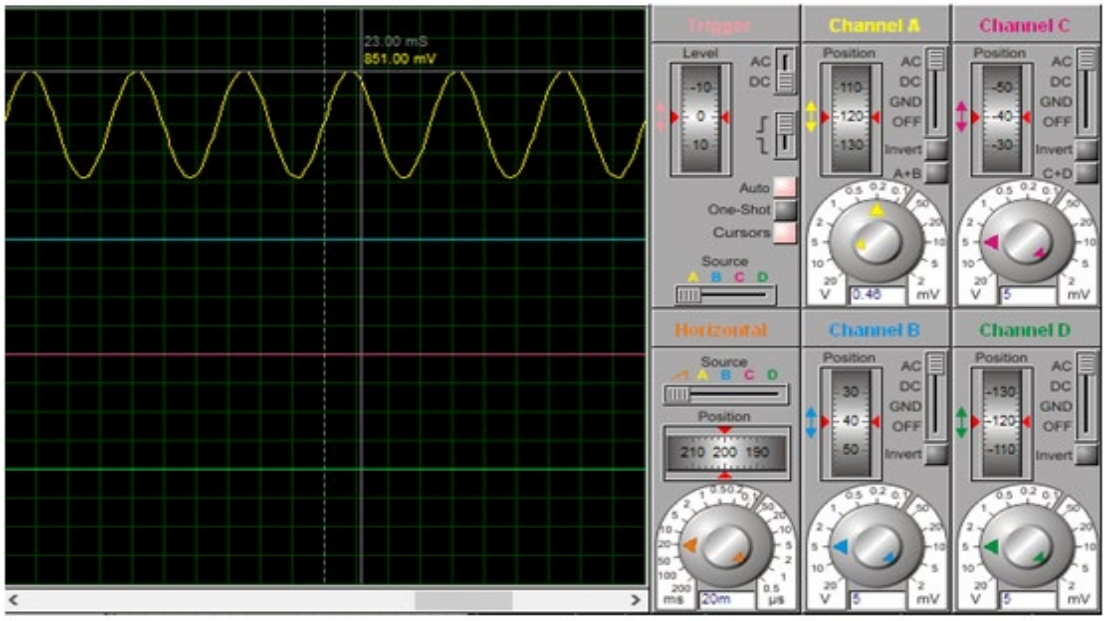
Para comprobar estos resultados se simulo el circuito del diagrama y se realizó un barrido de frecuencia, en las siguientes figuras se puede apreciar la respuesta temporal del filtro al hacer un barrido de frecuencia con señales de 3, 15, 30 Hz y 1V de amplitud donde se demuestra la atenuación que sufren las señales:

Figura 10. Respuesta del Filtro con Señal de 3Hz



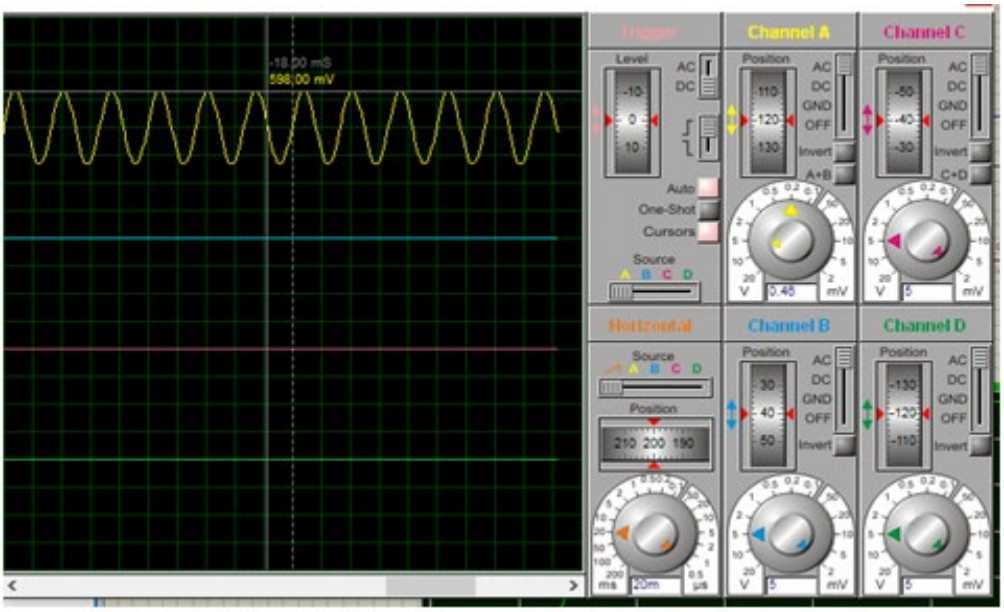
Elaborado por: Los Investigadores (2025).

Figura 11. Respuesta del Filtro con Señal de 15Hz



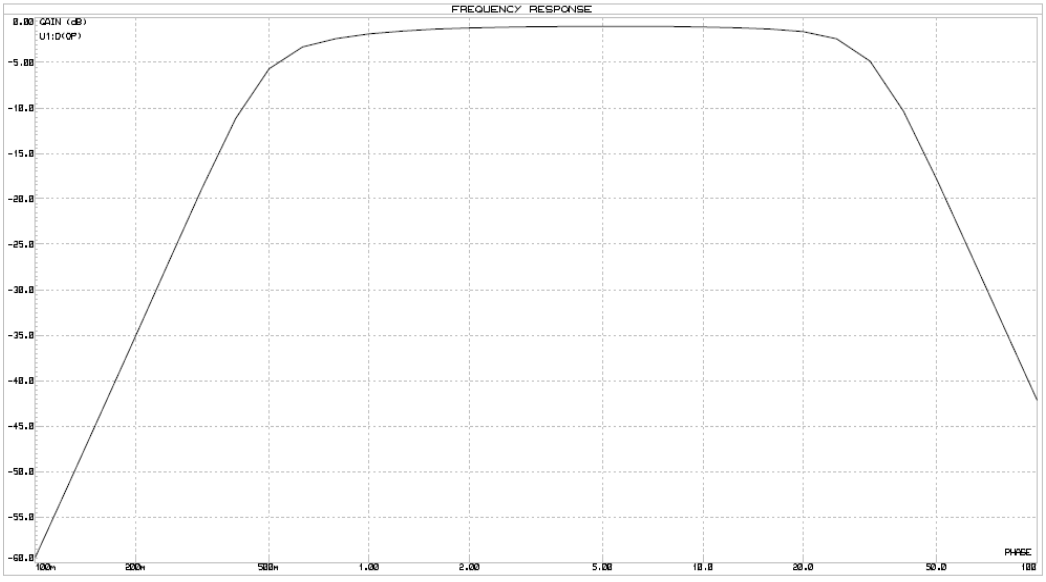
Elaborado por: Los Investigadores (2025).

Figura 12. Respuesta del Filtro con Señal de 30Hz



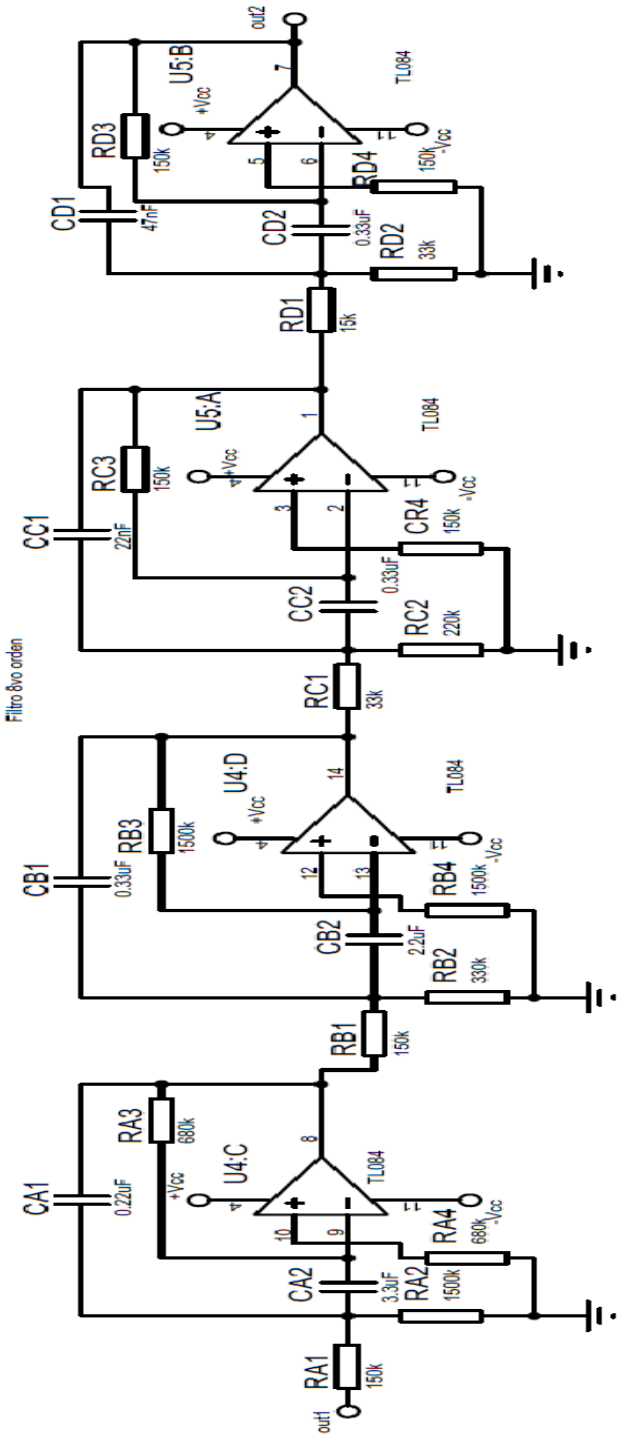
Elaborado por: Los Investigadores (2025).

Figura 13. Respuesta en Frecuencia del Filtro Simulado



Elaborado por: Los Investigadores (2025).

Figura 14. Circuito del Filtro Simulado



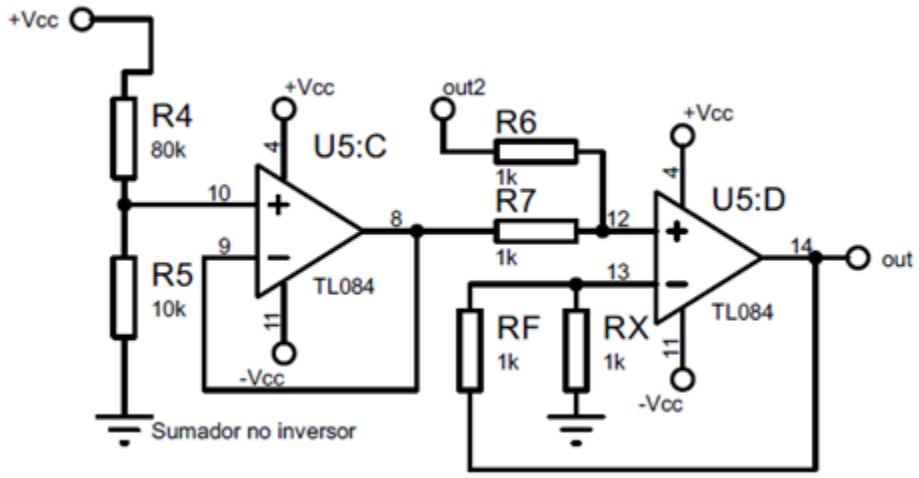
Elaborado por: Los Investigadores (2025).

Según los resultados obtenidos por los investigadores, se determinó que la respuesta en frecuencia del filtro es aceptable para esta aplicación y como tal, se dio por completada dicha etapa.

Cumplido los objetivos de las etapas anteriores, el siguiente paso fue adecuar las señales para que el CAD y el microcontrolador puedan trabajar con ellas; esto se logró agregando un offset de DC a las señales mediante un sumador no inversor unitario de tal forma que, la línea de base de la señal se desplace por encima de cero y de ese modo evitar el cruce por cero de la señal manteniéndola positiva.

Seguidamente, se usó el amplificador operacional TL084 cuya señal de entrada proviene del filtro y llega a su entrada no inversora, al mismo tiempo se hace pasar 1V de DC a dicha entrada del amplificador considerando que esta tensión es acoplada mediante un seguidor de tensión y de este modo se mantiene la señal en un rango de 0 a 2V, en la siguiente figura se puede apreciar el diagrama del circuito:

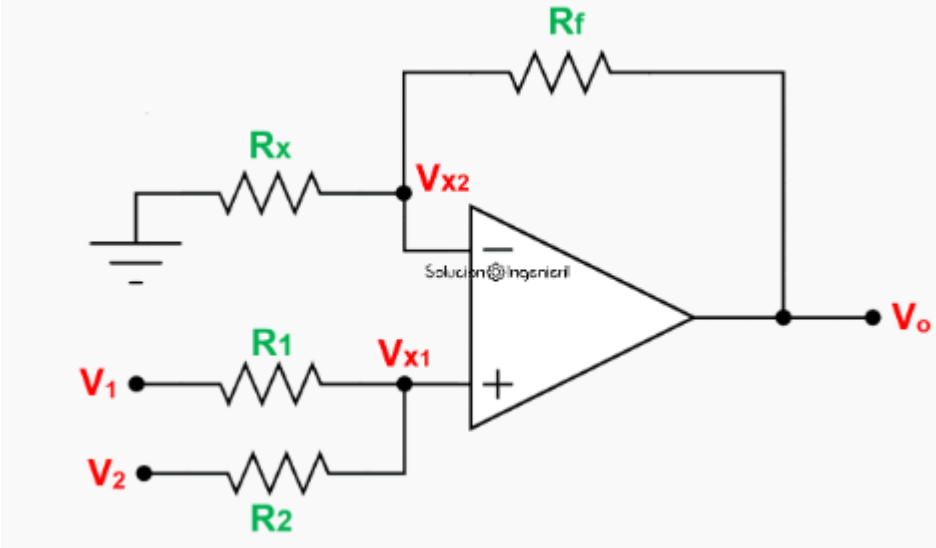
Figura 15. Diagrama del Sumador



Elaborado por: Los Investigadores (2025).

En esta figura se aprecia el seguidor de tensión que acopla el voltaje de DC al sumador y que en esta configuración todo el circuito es unitario en su ganancia, esto se demuestra considerando lo siguiente:

Figura 16. Circuito Sumador No Inversor



Fuente:https://solucioningenieril.com/amplificadores_operacionales/amplificador_sumador_no_inversor (2025).

Para hallar la relación entre las entradas y el voltaje de salida primero debe hallarse el voltaje que existe en ambas entradas del operacional en este caso son V_{x1} y V_{x2}, para hallar V_{x1} se consideran las entradas una a una mediante el teorema de superposición de modo que cuando V_2 es cero y se aplica un divisor de voltaje:

$$V_{x1} = V_1\left(\frac{R2}{R1 + R2}\right)$$

Y de igual manera cuando V_1 es cero:

$$V_{x1} = V_2\left(\frac{R1}{R1 + R2}\right)$$

Así V_{x1} es igual a:

$$V_{x1} = \frac{V_2 R1 + V_1 R2}{R1 + R2}$$

Para hallar V_{x2} se recurre a otro divisor de voltaje:

$$V_{x2} = V_O(\frac{RX}{RX + RF})$$

Considerando las propiedades de los amplificadores operacionales se considera que V_{x1} y V_{x2} son iguales por lo que:

$$\frac{V_2 R1 + V_1 R2}{R1 + R2} = V_O(\frac{RX}{RX + RF})$$

Despejando V_O se obtiene:

$$V_O = \frac{(RX + RF)(V_2 R1 + V_1 R2)}{RX(R1 + R2)}$$

La anterior formula demuestra la relación entre las entradas y la salida del amplificador para la configuración presentada, para esta propuesta se considera lo siguiente:
Si R = RX = RF = R1 = R2 entonces:

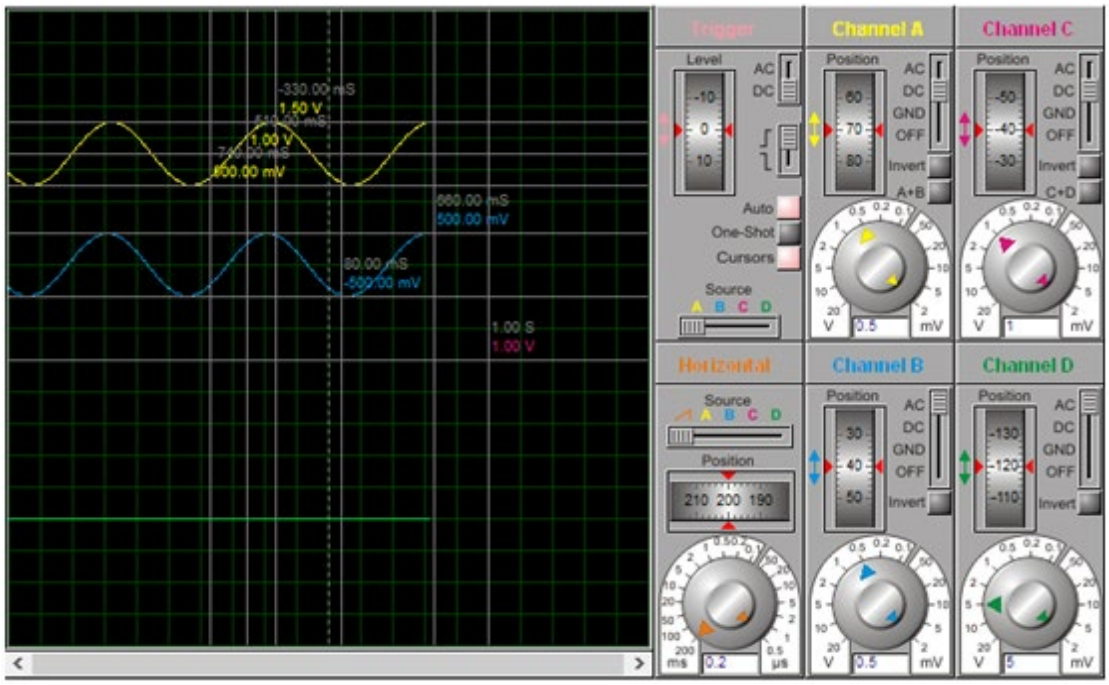
$$V_O = \frac{(R + R)(V_2 R + V_1 R)}{R(R + R)}$$

Y tras cancelar términos semejantes queda el siguiente resultado:

$$V_O = V_2 + V_1$$

Dado el resultado obtenido anteriormente, el mismo indica que para el circuito propuesto con una configuración en la que todas las resistencias son iguales, la salida es exactamente la sumatoria de las entradas sin amplificación; lo que es equivalente considerar, que su ganancia es igual a uno. En la siguiente figura se aprecian los resultados de la simulación del circuito:

Figura 17. Respuesta del Circuito Sumador

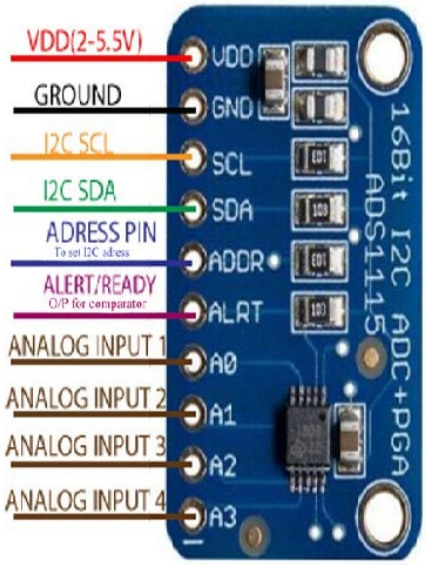


Elaborado por: Los Investigadores (2025).

Se observa a lo que respecta a la onda amarilla, representa la salida total del circuito, mientras que la onda azul es la señal que proviene del filtro y el punto rosa es el voltaje DC; se puede visualizar que la onda amarilla va desde quinientos mili-voltios hasta uno punto cinco voltios (500mV hasta 1.5V), por lo que el funcionamiento del sumador es apropiado y está listo para la implementación.

En concordancia con lo antes expuesto, se debe aclarar que una vez listas las etapas previamente descritas se procedió a configurar el CAD, se manejó para el prototipo el ADS1115, el cual es una conversor análogo-digital de 16bits de resolución compatible con el protocolo de comunicación I^2C y de bajo consumo de potencia, es apropiado para el desarrollo de instrumentación portátil y puede convertir datos a una tasa de hasta ochocientas sesenta muestras por segundo (860 SPS) y soporta señales desde más o menos doscientos cincuenta y seis mili-voltios (±256mV) hasta más o menos seis punto ciento cuarenta y cuatro voltios (±6.144V), permitiendo medidas precisas de señales pequeñas; en la siguiente figura se puede apreciar el componente y todos sus terminales:

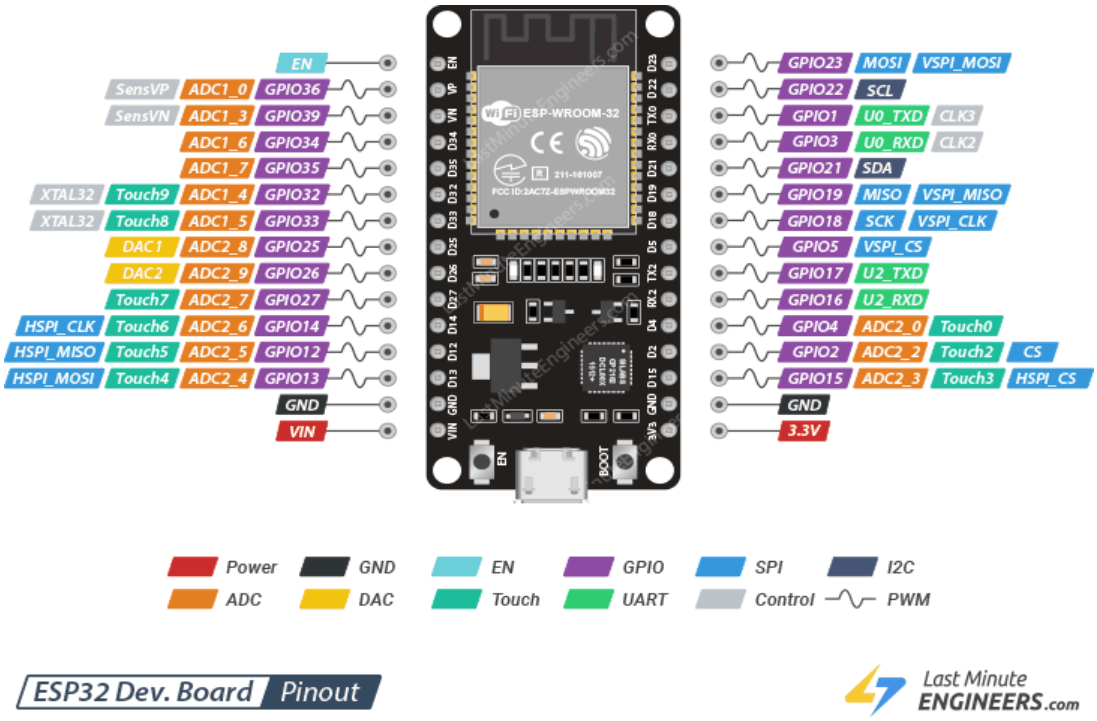
Figura 18. CAD ADS1115



Fuente:https://components101.com/modules/ads1115-module-with-programmable-gain-amplifier

Para controlar al ADS1115 se utilizó el microcontrolador ESP32, más específicamente la placa de desarrollo ESP32-D, aunque este microcontrolador cuenta con un CAD por sí mismo, se optó por usar el anteriormente descrito en virtud de que posee una mayor resolución y de este modo permite visualizar mejor las señales reconstruidas; en la siguiente figura se puede apreciar la placa de desarrollo usada junto con sus terminales:

Figura 19. Placa de Desarrollo ESP32.



Fuente: https://lastminuteengineers.com/esp32-pinout-reference/

Para manejar la programación del ESP32 se utilizó el IDE oficial de ESPRESSIF, en concreto la versión basada en Eclipse, de ese modo se puede programar, compilar y depurar con una interfaz que señala los errores, también permite a su vez controlar las especificaciones del hardware de un modo sencillo.

Bajo este contexto, cabe aclarar que para este prototipo el microcontrolador ESP32 se hará cargo de controlar el ADS1115, establecer un punto de acceso WiFi y manejar la comunicación mediante WebSockets con cualquier dispositivo que pueda acceder a un navegador Web; el control del ADS1115 se llevó a cabo utilizando una librería llamada "esp-idf-lib/ads111x", de este modo se simplificó la aplicación de las directrices de comunicación I^2C necesarias para el CAD; aunado a esto, se configuraron los

parámetros del mismo haciendo énfasis en que la adquisición de señales es en tiempo real por lo que el CAD debe trabajar de forma continua a su máxima velocidad de muestreo que en este caso son ochocientos sesenta muestra por segundo (860 SPS); para visualizar el código correspondiente referirse al ANEXO A.

El siguiente paso en el desarrollo, fue establecer un punto de acceso WiFi y un servidor con comunicación mediante WebSockets usando el ESP32, el punto de acceso es una red creada por el microcontrolador; la idea principal, es que cualquier dispositivo se conecte a dicha red, de modo que puedan hacer la petición al servidor que en éste caso es el ESP32, para así obtener la información; la petición se hace introduciendo la dirección IP del ESP32 en el navegador, de manera que el ESP32 envié la información que está captando el CAD ya procesada y diferenciada según los canales que tiene el prototipo de EEG.

Cabe destacar, que en el cliente deben visualizarse las ondas eléctricas cerebrales y dicha gráfica se realiza con el poder computacional del mismo dispositivo, esto es debido a que el microcontrolador sólo posee almacenado en su memoria la página web HTML y la librería para graficar sin conexión Uplot.iife.min.js y Uplot.min.css de manera que estos archivos son enviados para que se ejecuten según la solicitud. Se puede agregar, que debido a que se está usando comunicación mediante WebSocket el microcontrolador puede manejar las solicitudes de máximo dos dispositivos, la grafica se puede visualizar en el dominio del tiempo y en el dominio de la frecuencia; en el tiempo se aprecia la magnitud contra el tiempo funcionando de manera similar a un osciloscopio, graficando los puntos mientras los va recibiendo, esto se logra con el uso de la librería uPlot. La grafica del dominio de la frecuencia representa la magnitud del espectro contra la frecuencia, por lo que se aprecia donde esta distribuida la mayor cantidad de energía entre los armónicos de la señal graficada, para lograr esto se implemento un algoritmo FFT Cooley-Tukey iterativo que permite trabajar con

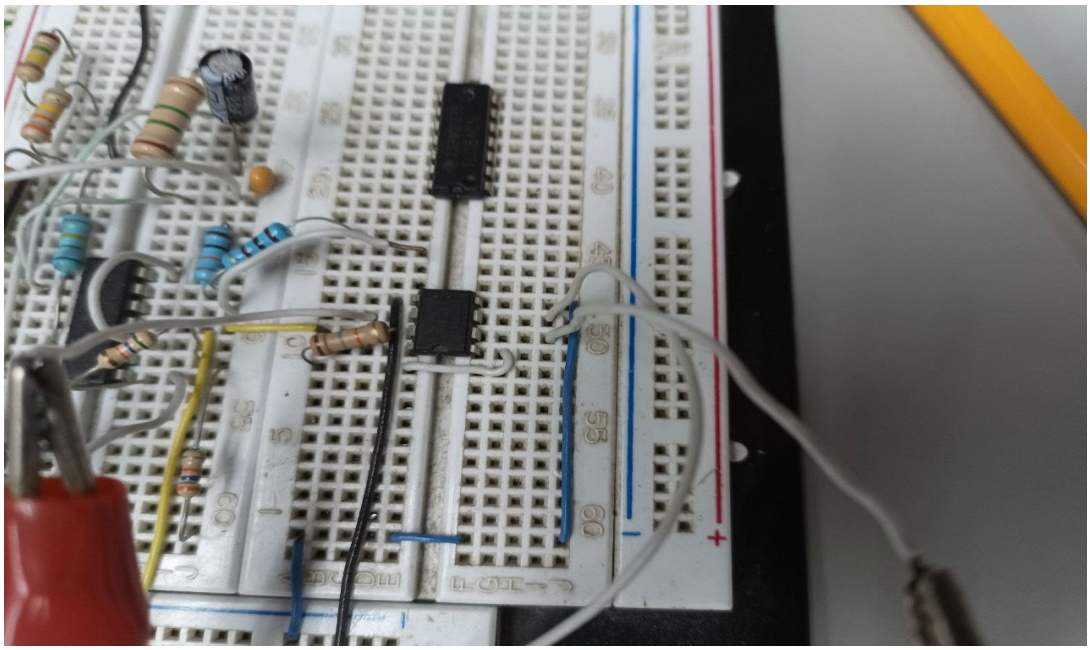
tamaños de transformada en potencias de dos con un límite de 2048 muestras. Para mayor claridad de lo anteriormente expuesto referirse al ANEXO B.

4.1. RESULTADOS DE LAS PRUEBAS DEL PROTOTIPO

Las pruebas del prototipo se realizaron por etapas al igual que la simulación; es decir, se empezó por el amplificador de bio-potenciales, seguido por el amplificador de ganancia, el filtro pasa-banda de 8vo orden y por último el sumador no inversor. Las pruebas se pueden apreciar en las siguientes figuras, destacando que éstas sólo comprueban el funcionamiento de las configuraciones expuestas; por lo tanto, no comparten los mismos valores de voltaje o resistencia en relación a la simulación.

Obsérvese a continuación la figura 20, la cual hace referencia al amplificador de instrumentación.

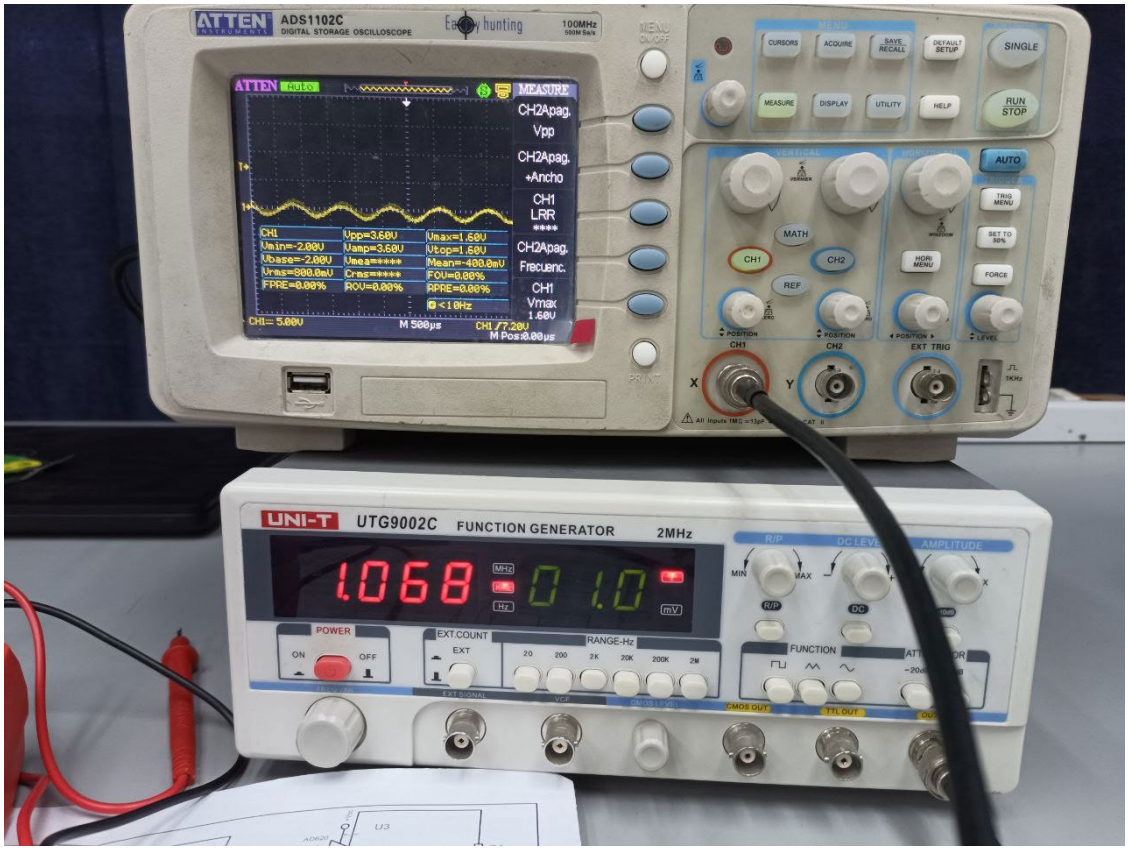
Figura 20. Amplificador de Instrumentación



Elaborado por: Los Investigadores (2026).

Seguidamente, se muestra la figura 21, la cual hace referencia a la medida del amplificador de instrumentación; para esta prueba el amplificador tenía una ganancia de aproximadamente 2.2, de manera que con un voltaje de entrada de 1V se puede esperar a la salida un voltaje de 2.2V; se puede apreciar en el osciloscopio de la figura 21, que los valores son lo suficientemente cercanos para ser aceptables.

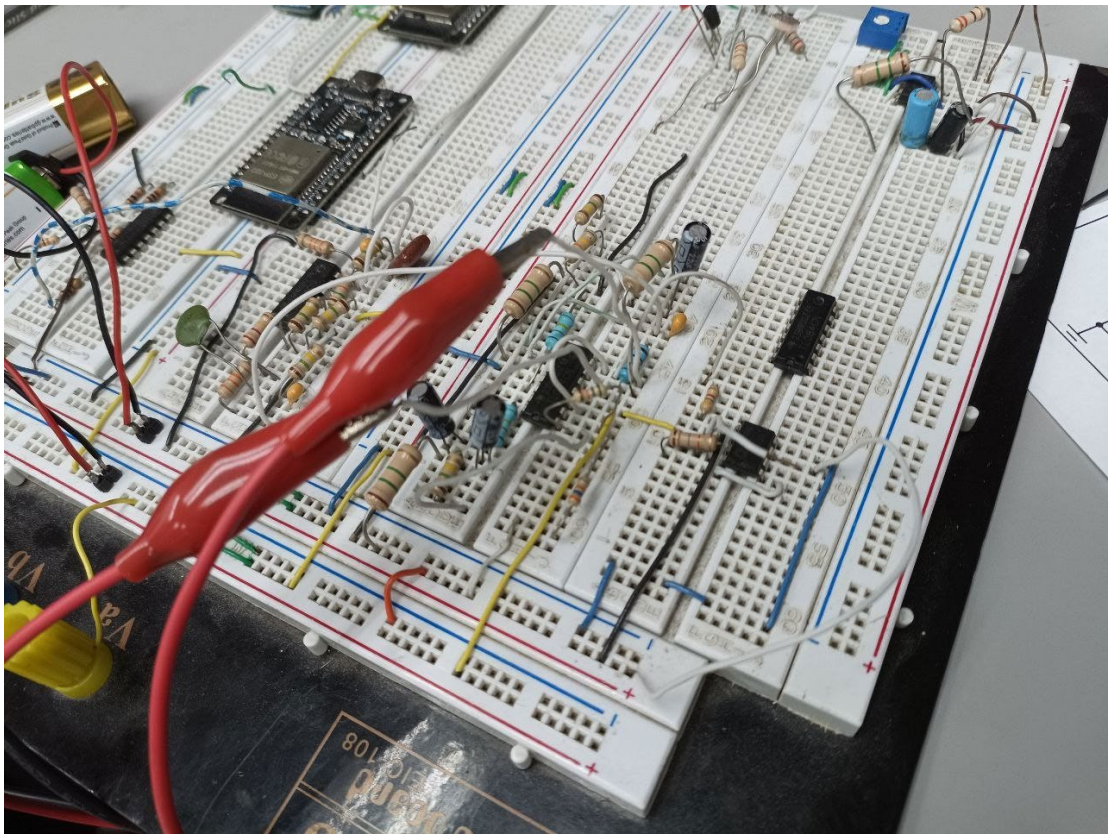
Figura 21. Medida del Amplificador de Instrumentación



Elaborado por: Los Investigadores (2026).

Dentro de esta línea descriptiva, se aprecia en la figura 22 el circuito que corresponde al amplificador de ganancia.

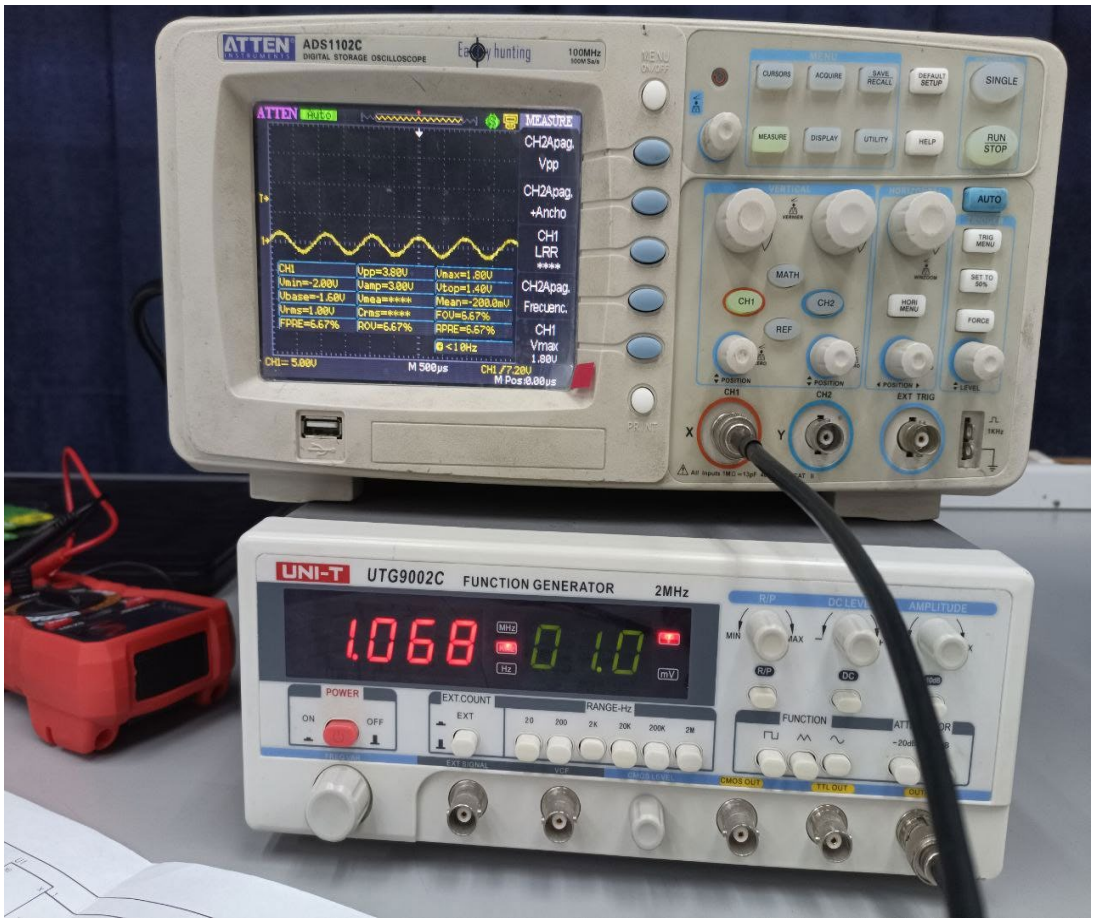
Figura 22. Amplificador de Ganancia



Elaborado por: Los Investigadores (2026).

Es importante destacar, que se procedió a configurar el amplificador de ganancia para tener una ganancia de aproximadamente 2 y en relación al osciloscopio que se muestra en la figura 23, se puede apreciar que la señal tiene los valores apropiados para su correcto funcionamiento.

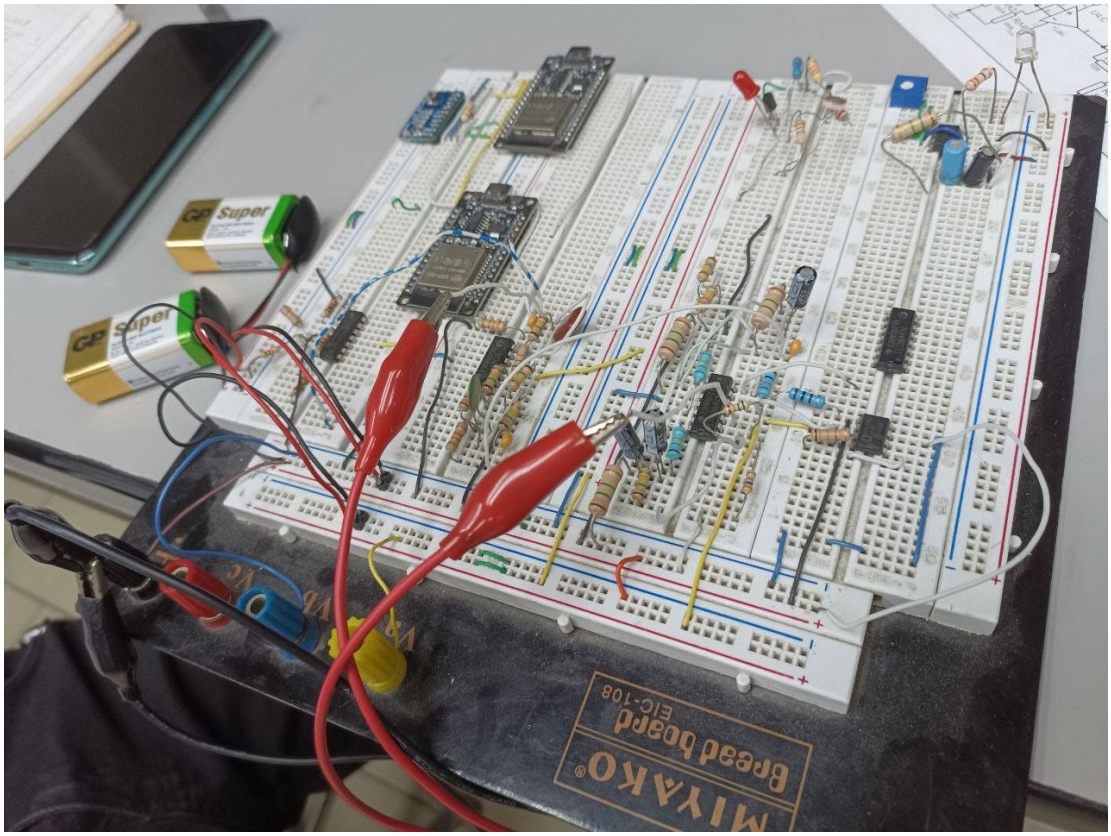
Figura 23. Medida del Amplificador de Ganancia



Elaborado por: Los Investigadores (2026).

Se muestra a continuación la figura 24, correspondiente al circuito del filtro pasa-banda de 8vo orden:

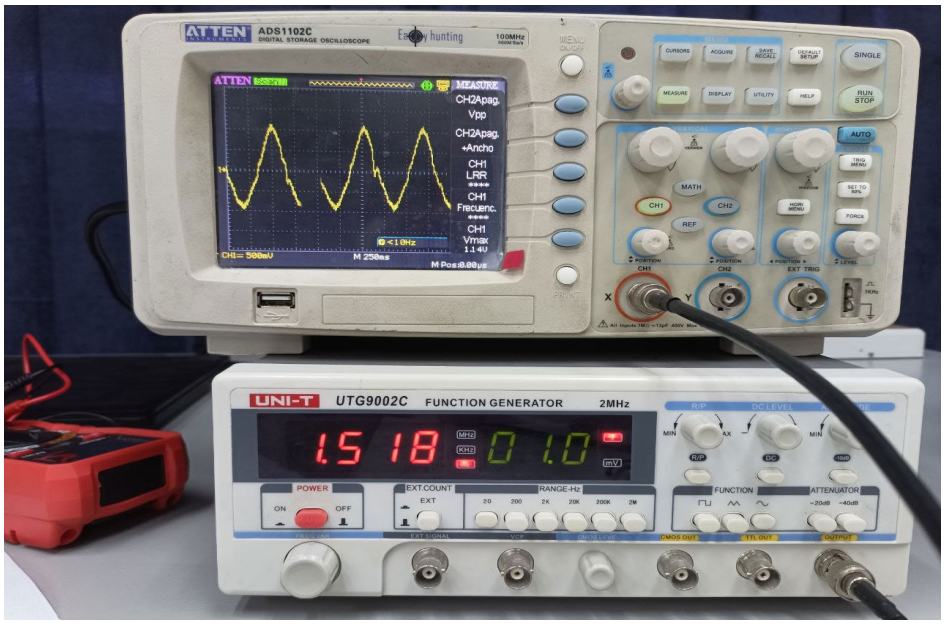
Figura 24. Filtro Pasa-Banda de 8vo Orden



Elaborado por: Los Investigadores (2026).

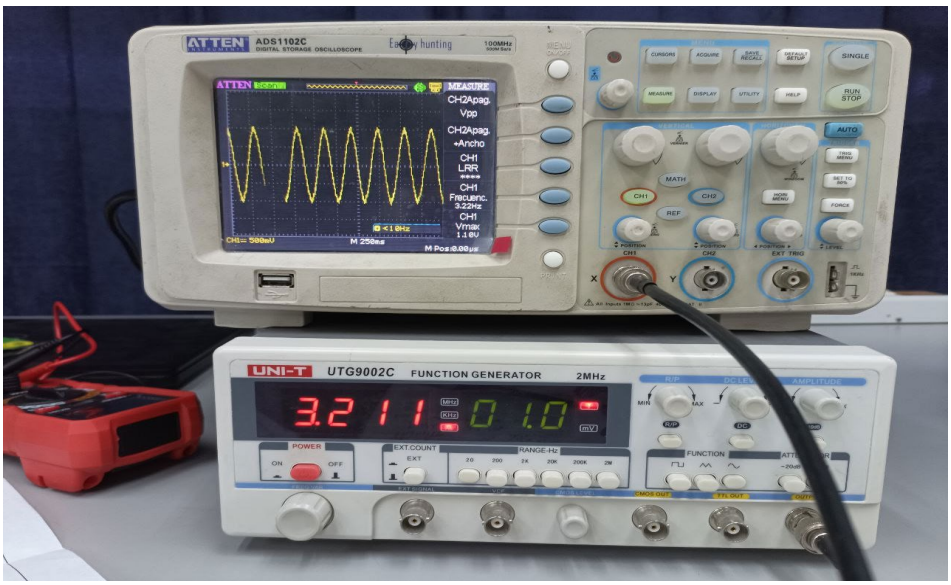
Para comprobar el funcionamiento del filtro, se hizo un barrido de frecuencia considerando que sus frecuencias de corte son 0.5 y 30 Hz para la baja y alta respectivamente, en lo que corresponde a los valores de frecuencia del barrido fueron 1.5, 3, 6, 12, 20 y 30 Hz, esto se puede observar de manera más clara desde la figura 25 hasta la figura 30, las mismas se detallan a continuación:

Figura 25. Filtro Pasa-Banda de 8vo Orden con Señal de 1.5Hz



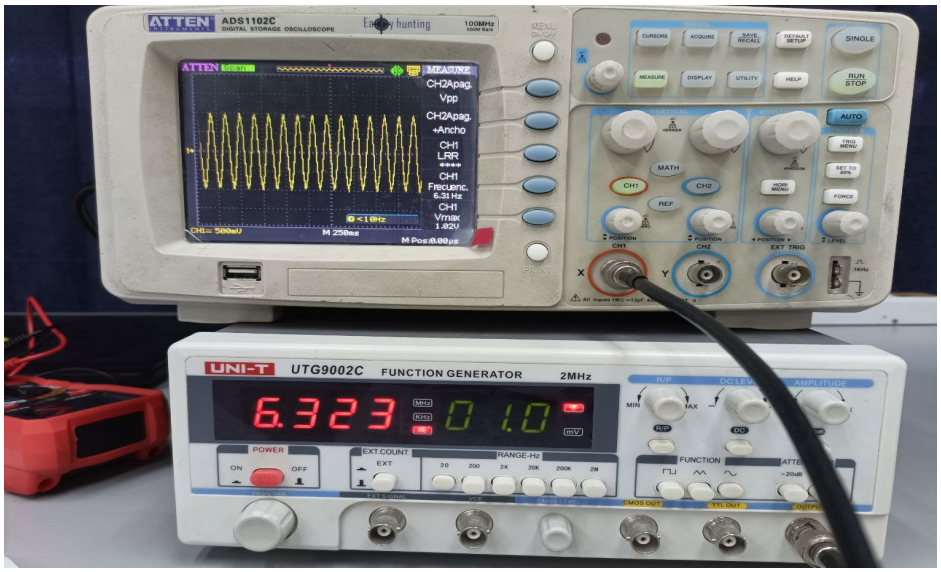
Elaborado por: Los Investigadores (2026).

Figura 26. Filtro Pasa-Banda de 8vo Orden con Señal de 3Hz



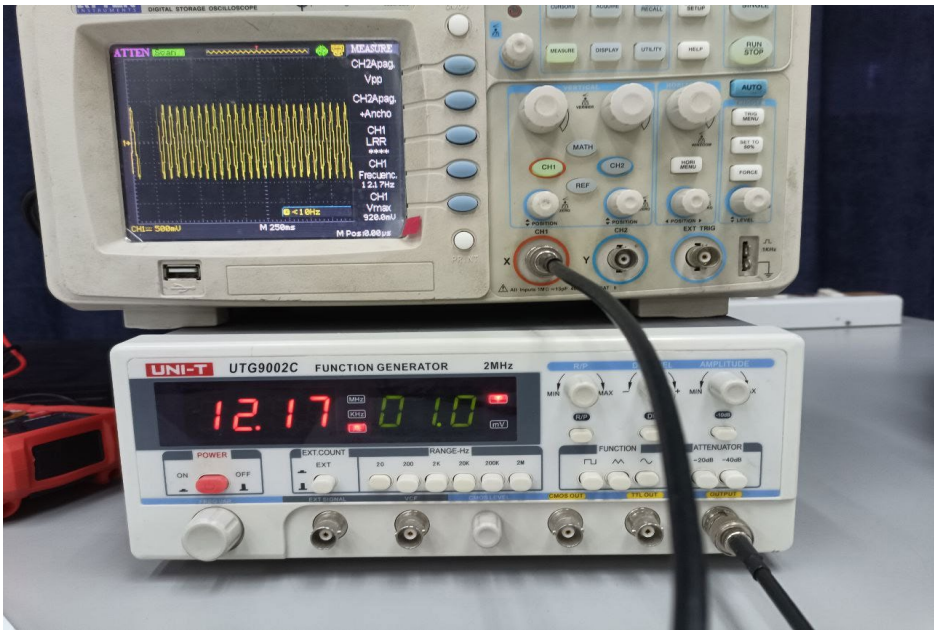
Elaborado por: Los Investigadores (2026).

Figura 27. Filtro Pasa-Banda de 8vo Orden con Señal de 6Hz



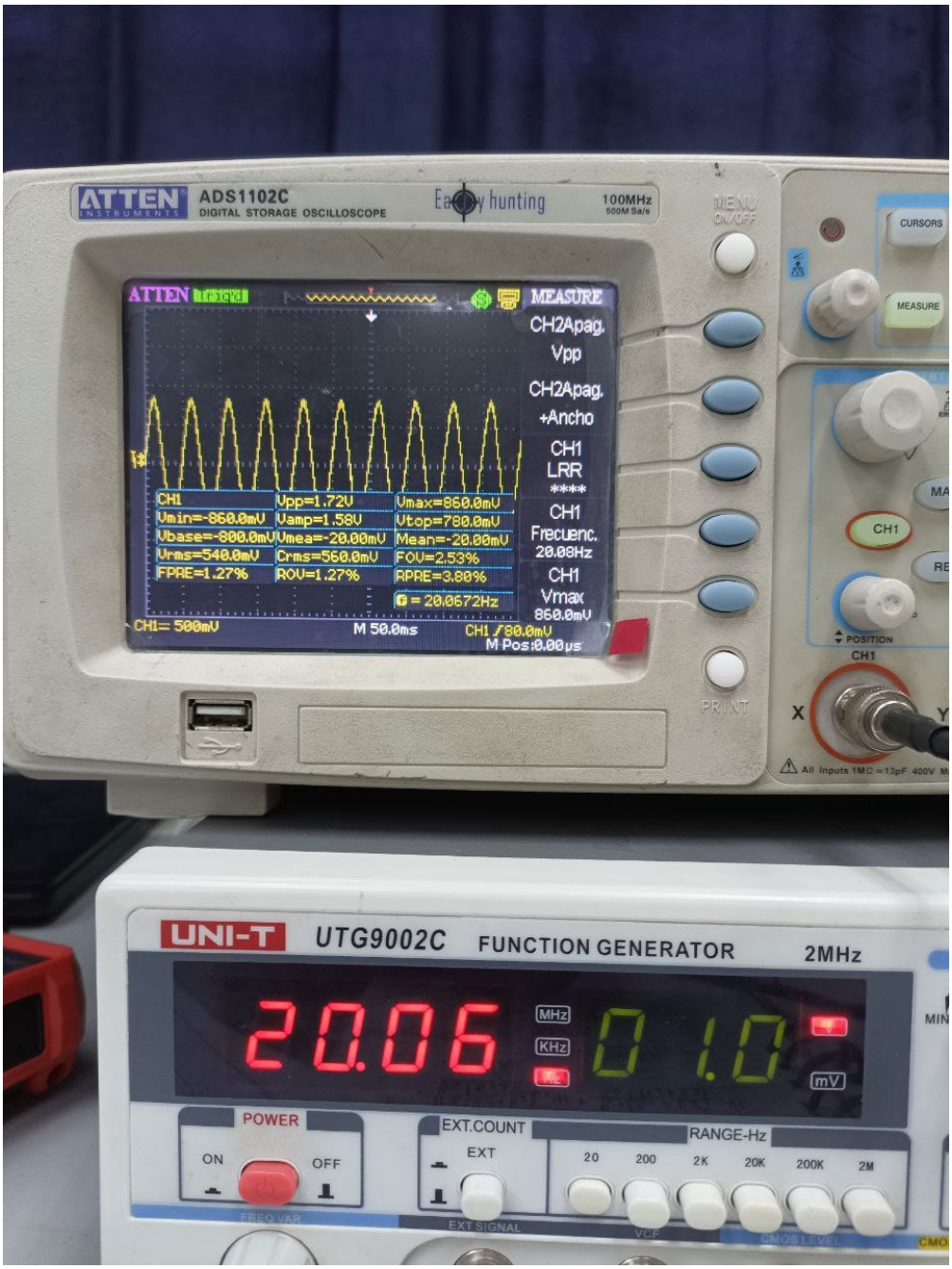
Elaborado por: Los Investigadores (2026).

Figura 28. Filtro Pasa-Banda de 8vo Orden con Señal de 12Hz



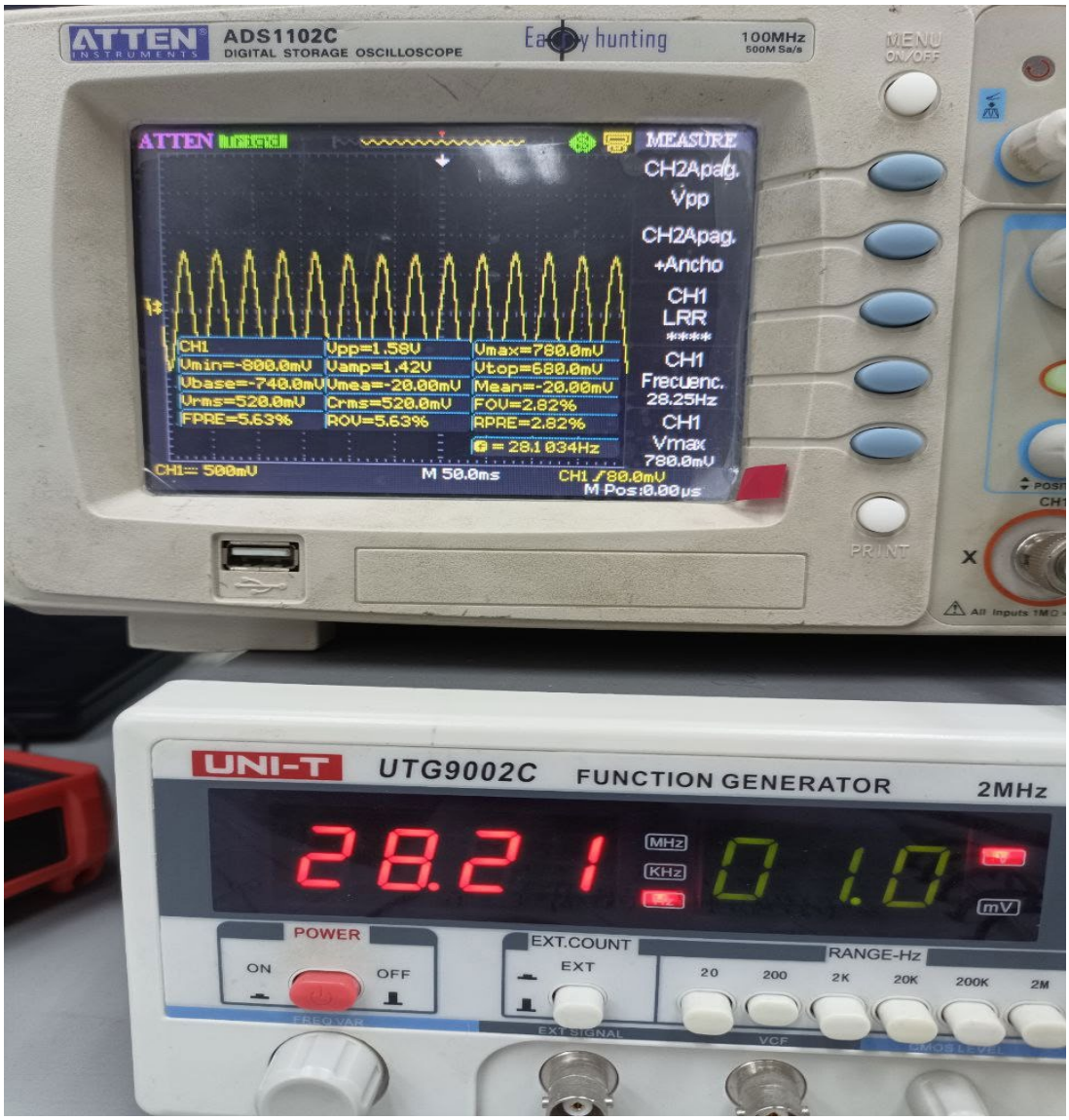
Elaborado por: Los Investigadores (2026).

Figura 29. Filtro Pasa-Banda de 8vo Orden con Señal de 20Hz



Elaborado por: Los Investigadores (2026).

Figura 30. Filtro Pasa-Banda de 8vo Orden con Señal de 30Hz

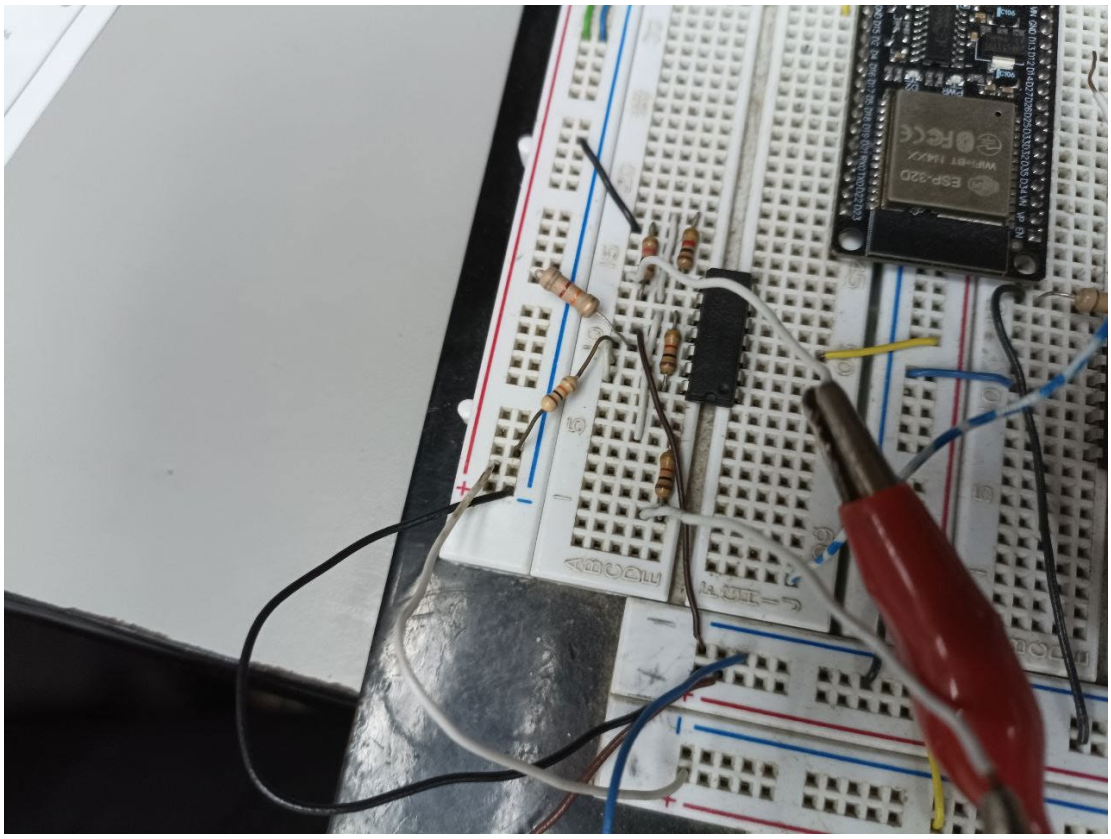


Elaborado por: Los Investigadores (2026).

Se observar en las diferentes figuras del filtro expuestas anteriormente, que el voltaje de señal de salida disminuye conforme se acerca a los límites del filtro comprobando así de este modo su funcionamiento.

Seguidamente, dentro de este contexto se observa en la figura 31 lo concerniente al circuito del sumador no inversor:

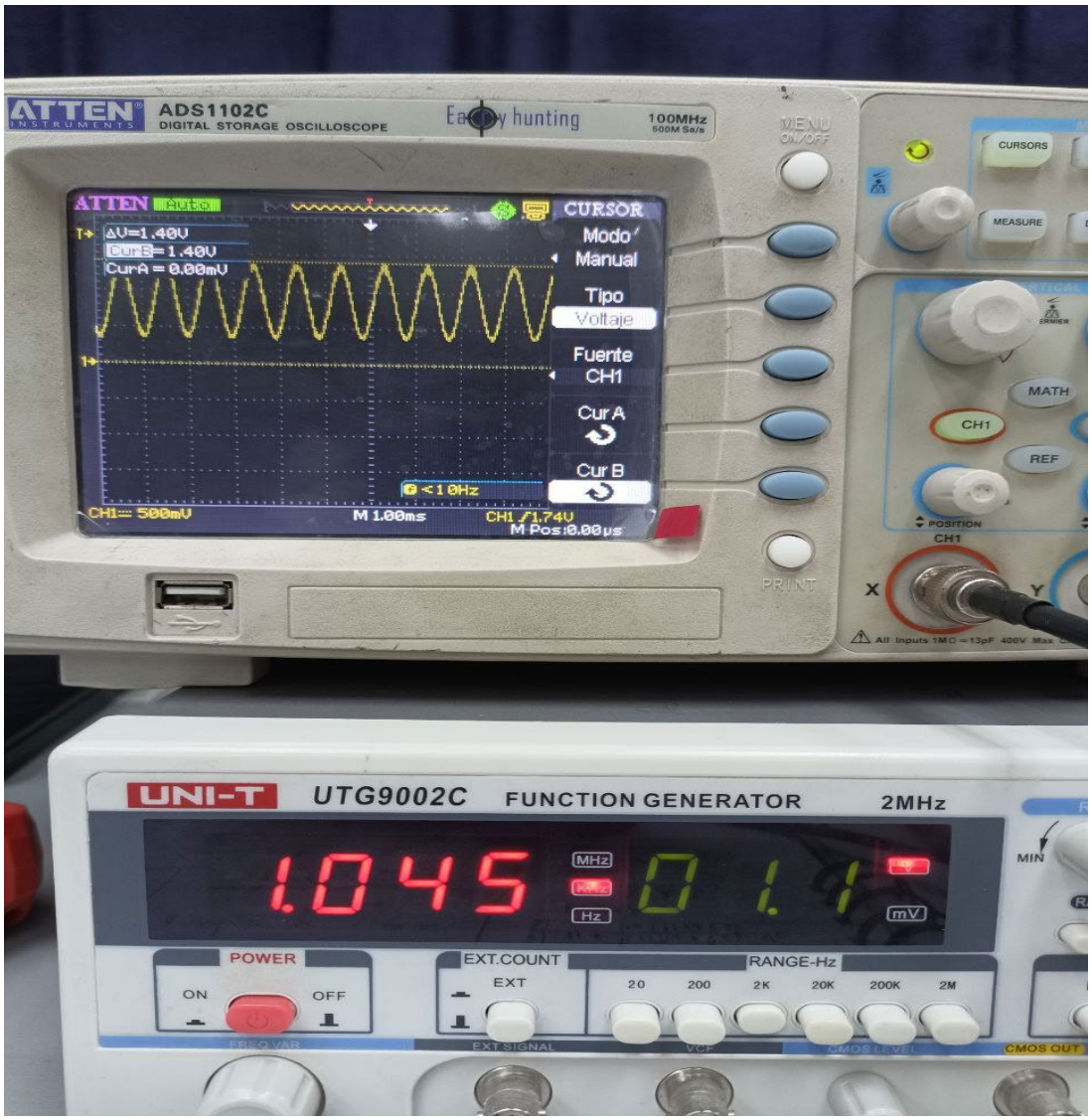
Figura 31. Sumador no Inversor



Elaborado por: Los Investigadores (2026).

Se puede apreciar en la figura 32, que el sumador cumple correctamente su función, esto se debe a que como se le está alimentando con una señal de 1Vpp y una señal de 1Vdc éstas se suman y la línea de base de la señal procede a desplazarse en un voltio positivo, que visualmente se aprecia en el osciloscopio como un desplazamiento hacia arriba.

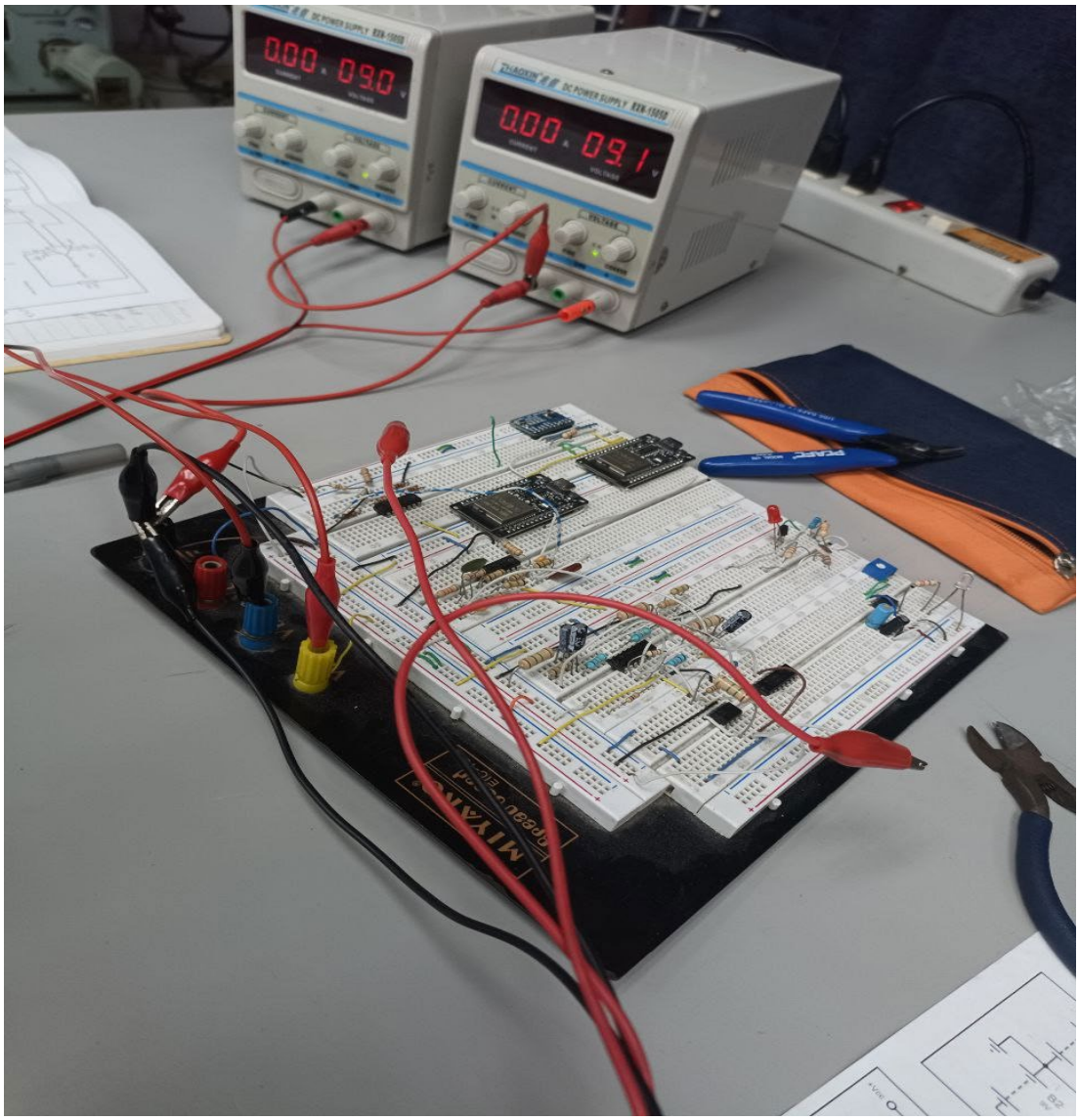
Figura 32. Medida del Sumador no Inversor



Elaborado por: Los Investigadores (2026).

Una vez comprobado todas las etapas del circuito analógico por separado, se procede a evaluar su desempeño de manera conjunta; en sentido, se muestra en la figura 33 el respectivo circuito analógico completo:

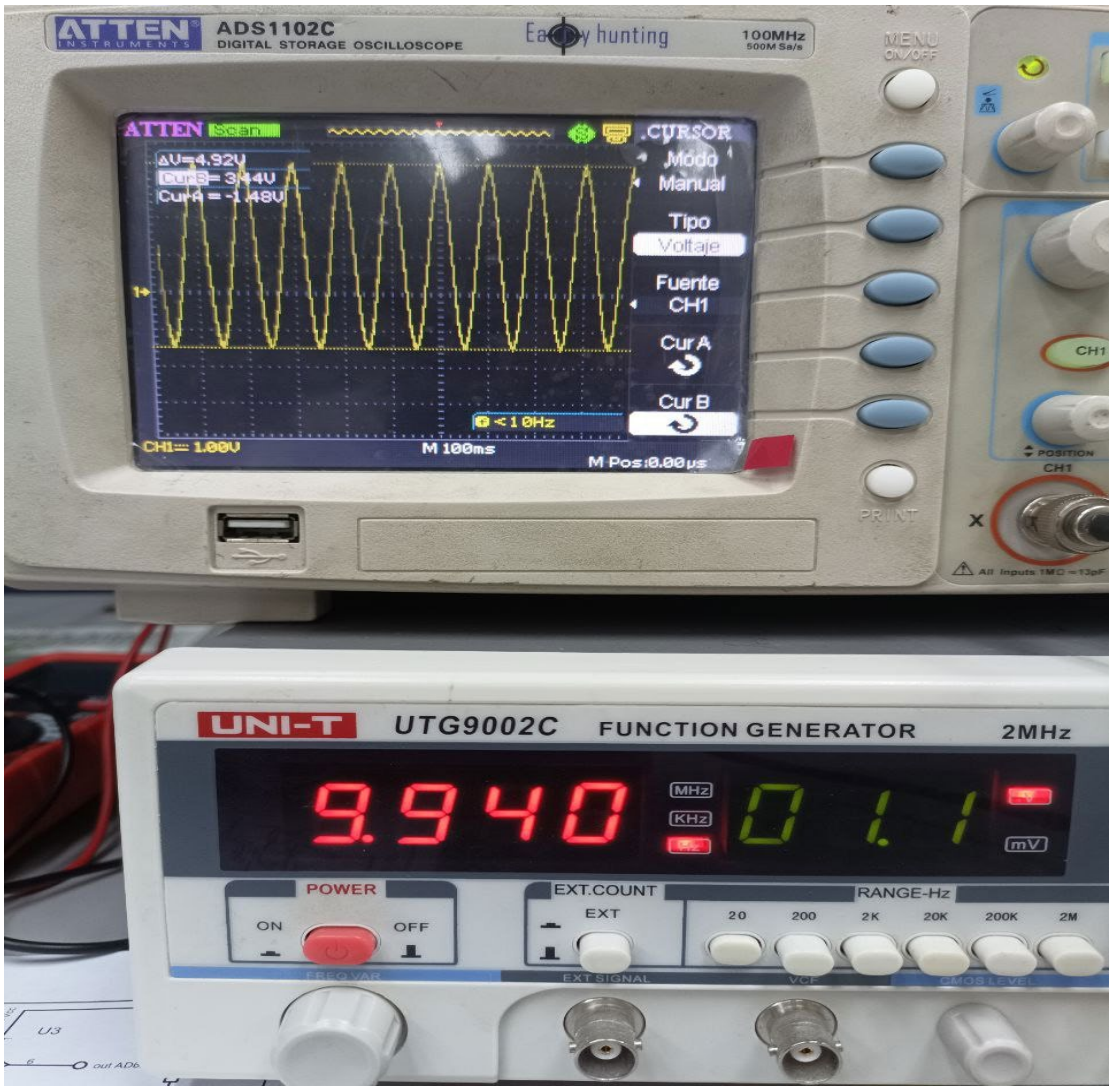
Figura 33. Circuito Analógico Completo



Elaborado por: Los Investigadores (2026).

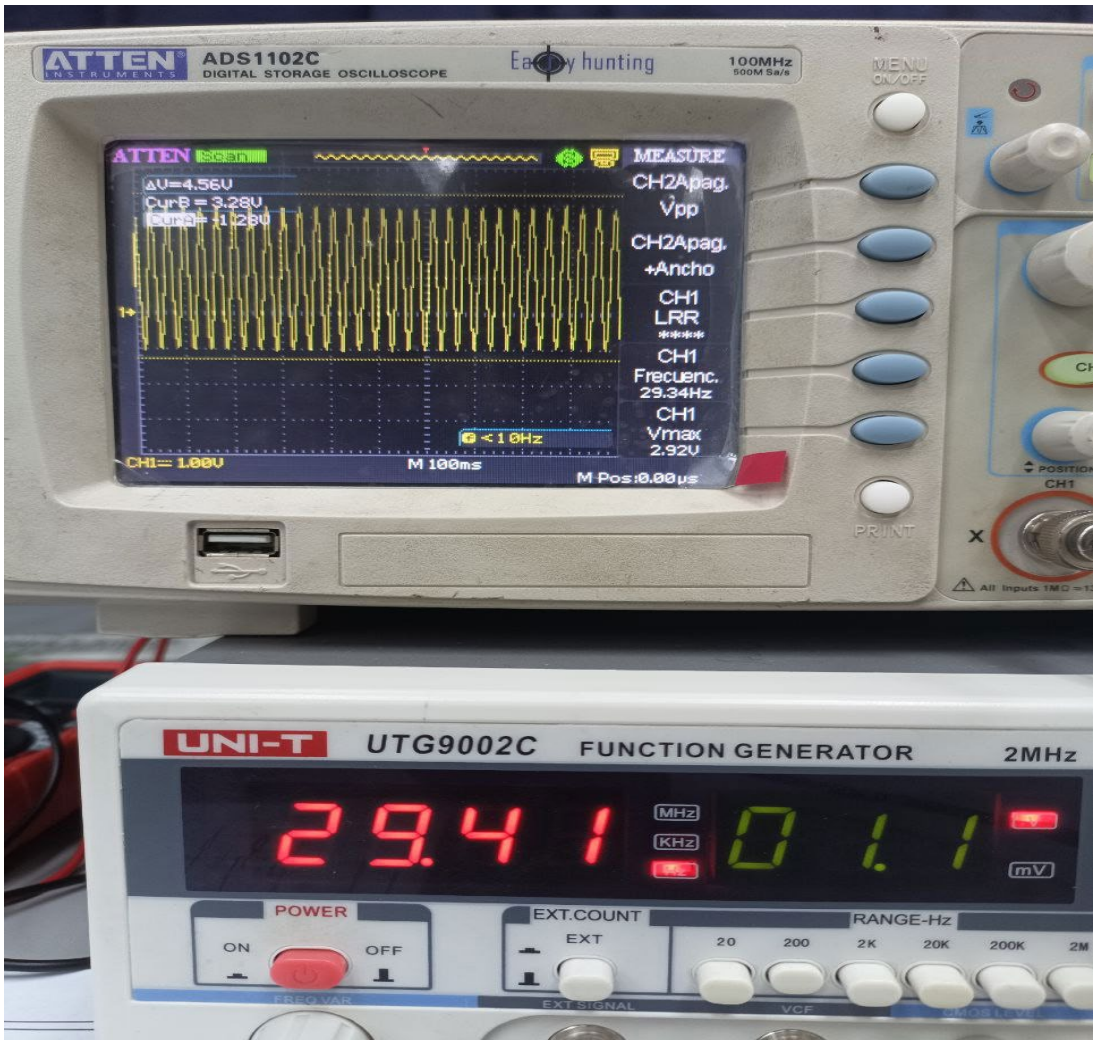
Se procede a mostrar las pruebas del circuito analógico completo con señales de 10 Hz y 30Hz correspondiente a las siguientes figuras:

Figura 34. Prueba del Circuito Analógico Completo con Señal de 10Hz



Elaborado por: Los Investigadores (2026).

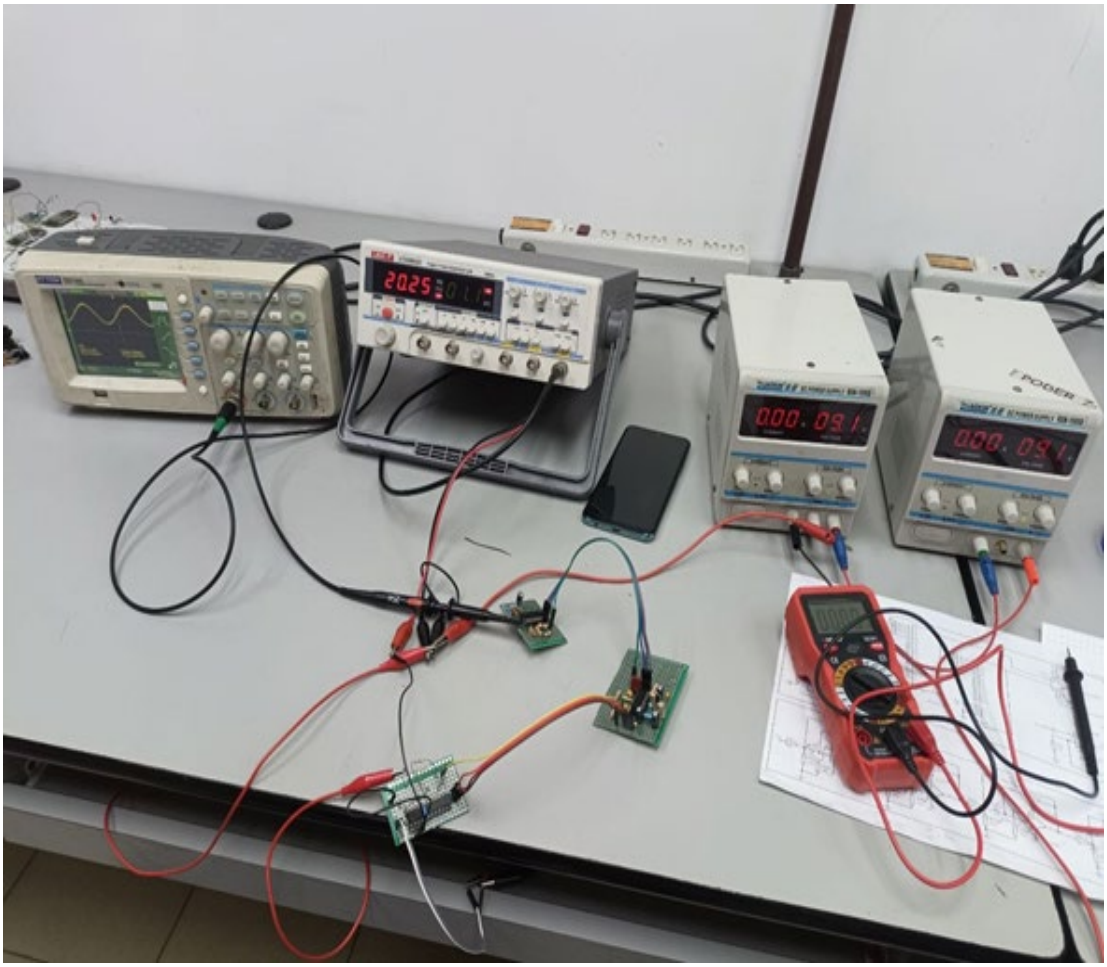
Figura 35. Prueba del Circuito Analógico Completo con Señal de 30Hz



Elaborado por: Los Investigadores (2026)

En las figuras 35 y 36 se aprecia que la señal esta desplazada hacia el lado positivo, que la amplitud de la señal disminuye conforme la frecuencia se acerca a 30 Hz y que la señal esta amplificada, lo que a grandes rasgos corrobora que el circuito trabaja adecuadamente en su conjunto y es capaz de acondicionar las señales de entrada para que el microcontrolador las procese.

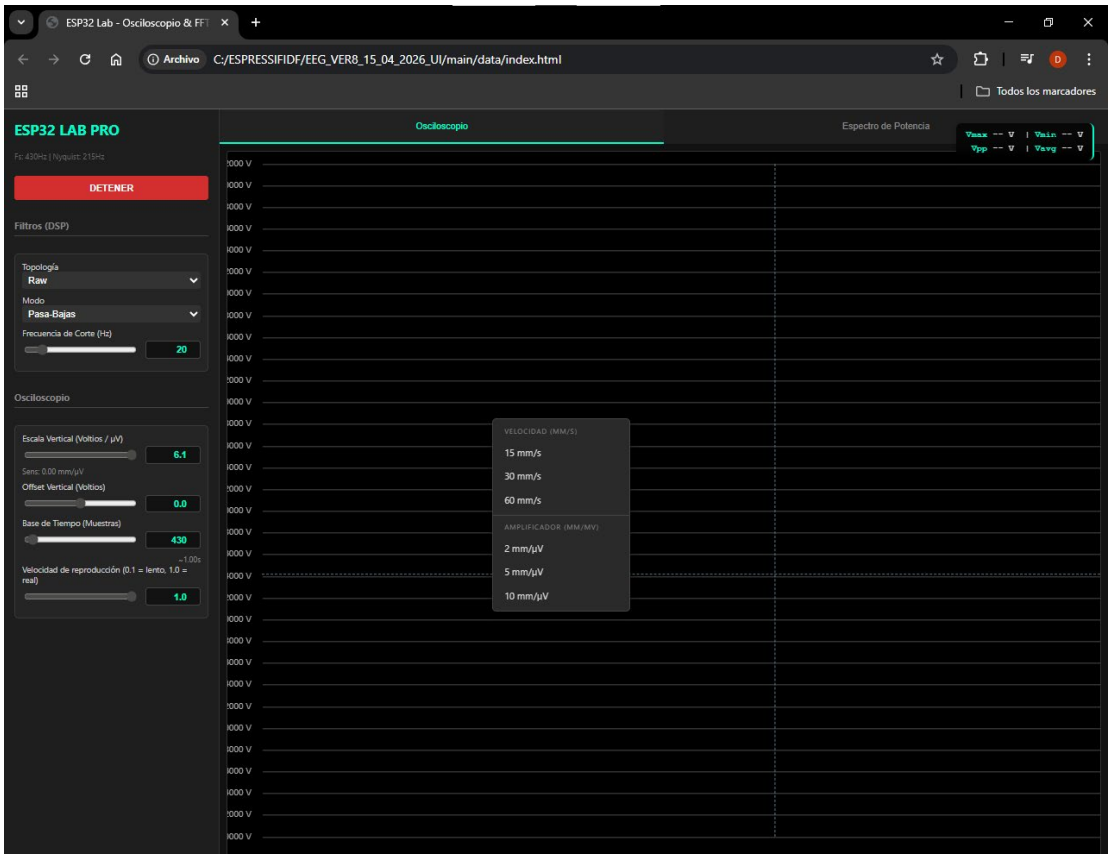
Figura 36. Circuito de Adquisición de Datos Finalizados



Elaborado por: Los Investigadores (2026)

Una vez comprobado el circuito de adquisición de datos se continuo con su montaje en unas baquelitas y posteriormente se corroboró que todo estuviese en orden repitiendo las pruebas descritas anteriormente.

Figura 37. Interfaz de la Pagina Web

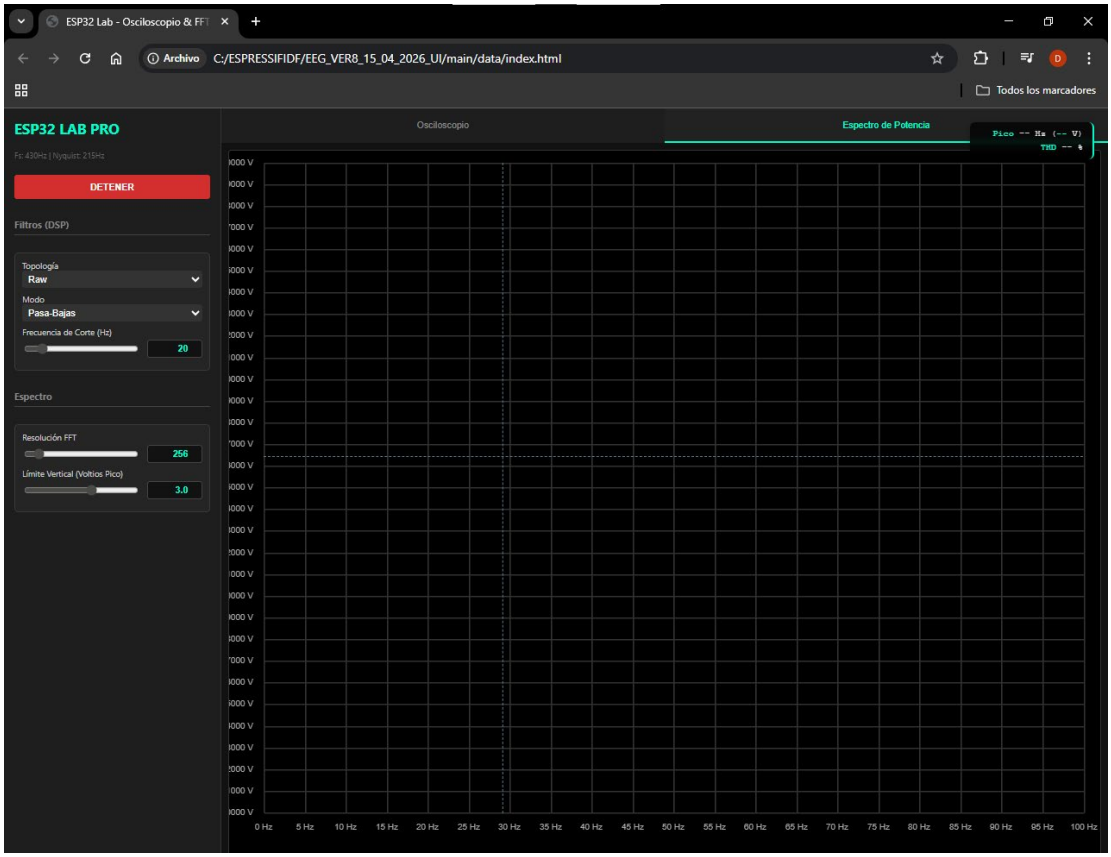


Elaborado por: Los Investigadores (2026)

En la figura 37 se puede apreciar la interfaz de la pagina web desarrollada, en la esquina superior izquierda se tiene un botón que permite detener la grafica, justo debajo esta un cuadro que enmarca los controles del filtrado digital, el que marca topología permite elegir entre filtro FIR, IIR y Raw que es el valor crudo y por tanto no procesado de los datos, el botón que marca Modo permite elegir entre si el filtro es pasa-bajas o pasa-altas y el botón deslizante denominado Frecuencia de Corte, permite elegir la frecuencia del corte del filtro. Debajo de los controles del filtrado, se encuentra el cuadro de los controles del osciloscopio, todos estos son controles deslizantes que permiten variar la escala vertical, el offset vertical, la base de tiempo o numero de muestras visualizadas y la velocidad de reproducción en caso de que la señal se vea

muy acelerada, además si se pulsa click derecho se abre un menú deslizante con opciones ya establecidas según los estándares para los electroencefalogramas, dichas opciones varian el eje horizontal y vertical, en la parte superior se encuentran los botones para seleccionar la grafica, pudiendo elegir entre osciloscopio y espectro de potencia.

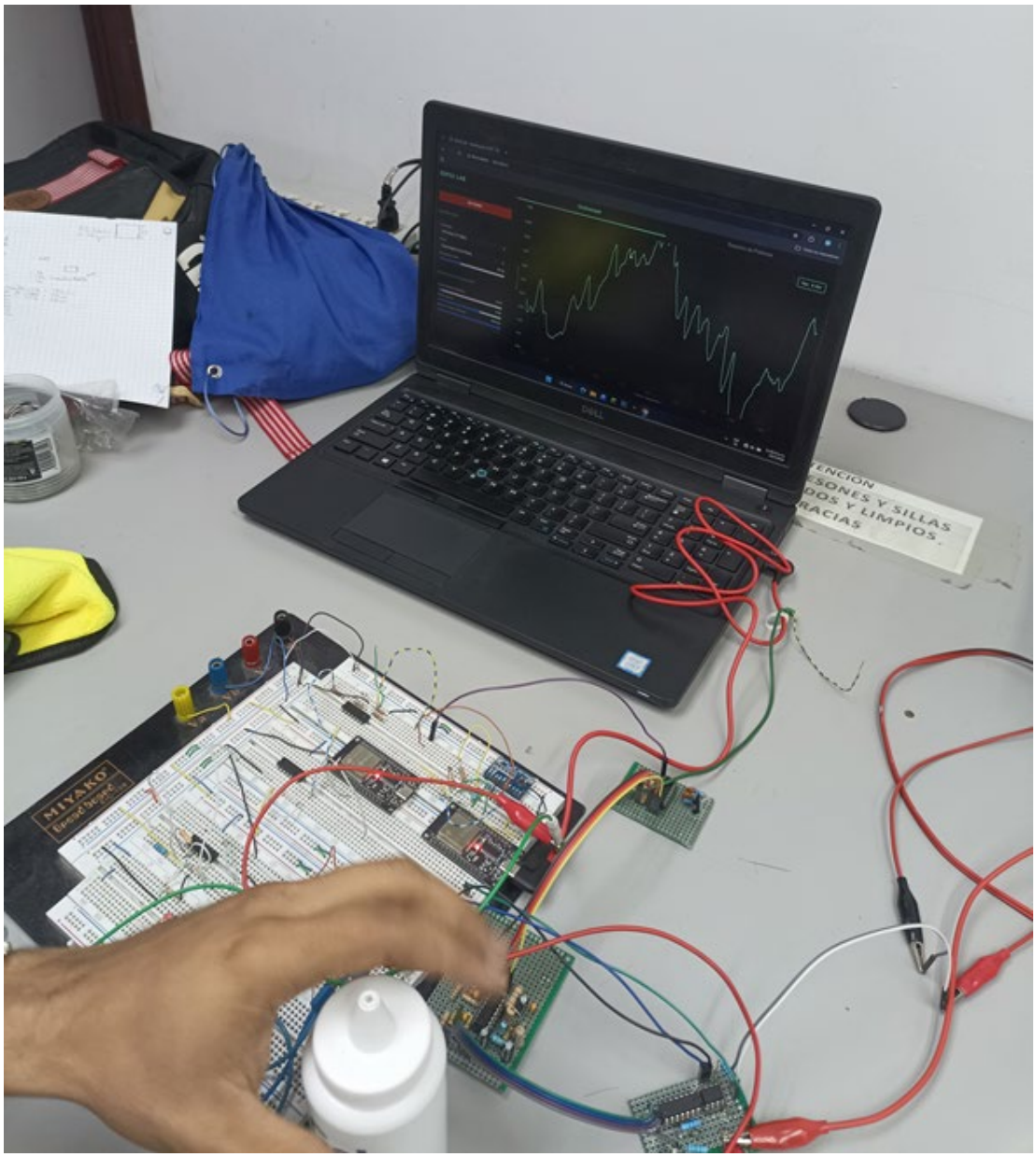
Figura 38. Interfaz de la Pagina Web (Cont.)



Elaborado por: Los Investigadores (2026)

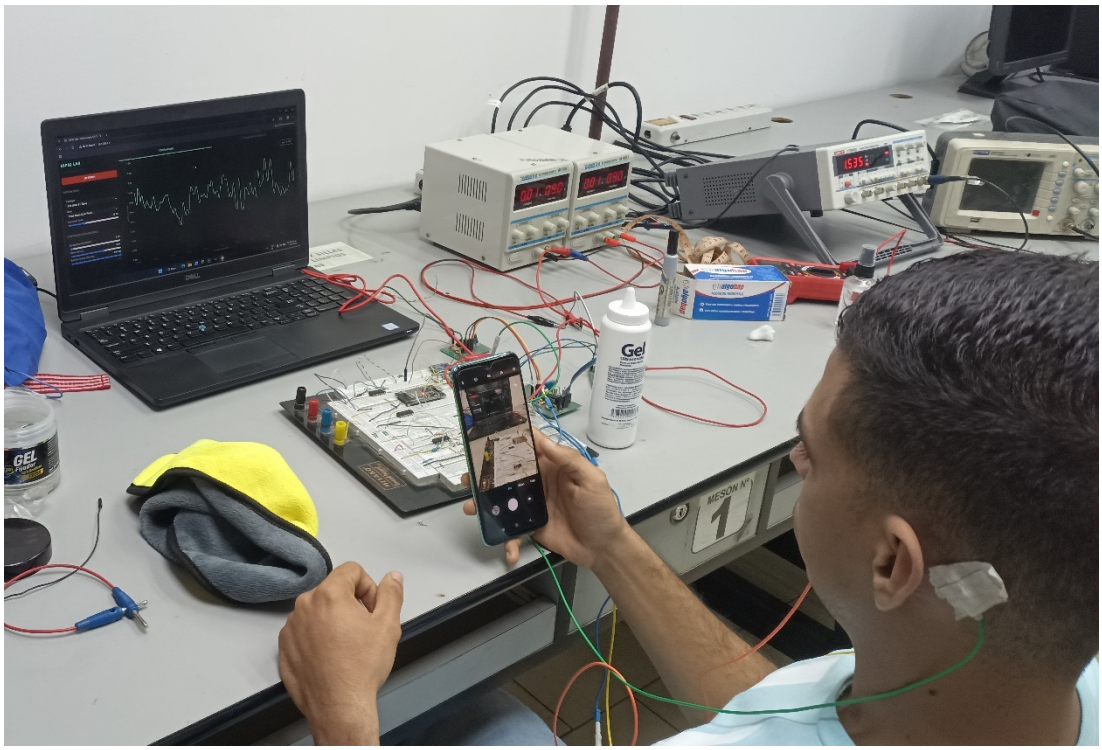
En la figura 38 se pueden apreciar los controles de la interfaz para la grafica del espectro de potencia, la resolución FFT determina la precisión del calculo de este algoritmo, la escala vertical ajusta la magnitud del voltaje visualizado.

Figura 39. Resultados en el Tiempo del Prototipo



Elaborado por: Los Investigadores (2026)

Figura 40. Resultados en el Tiempo del Prototipo (Cont.)



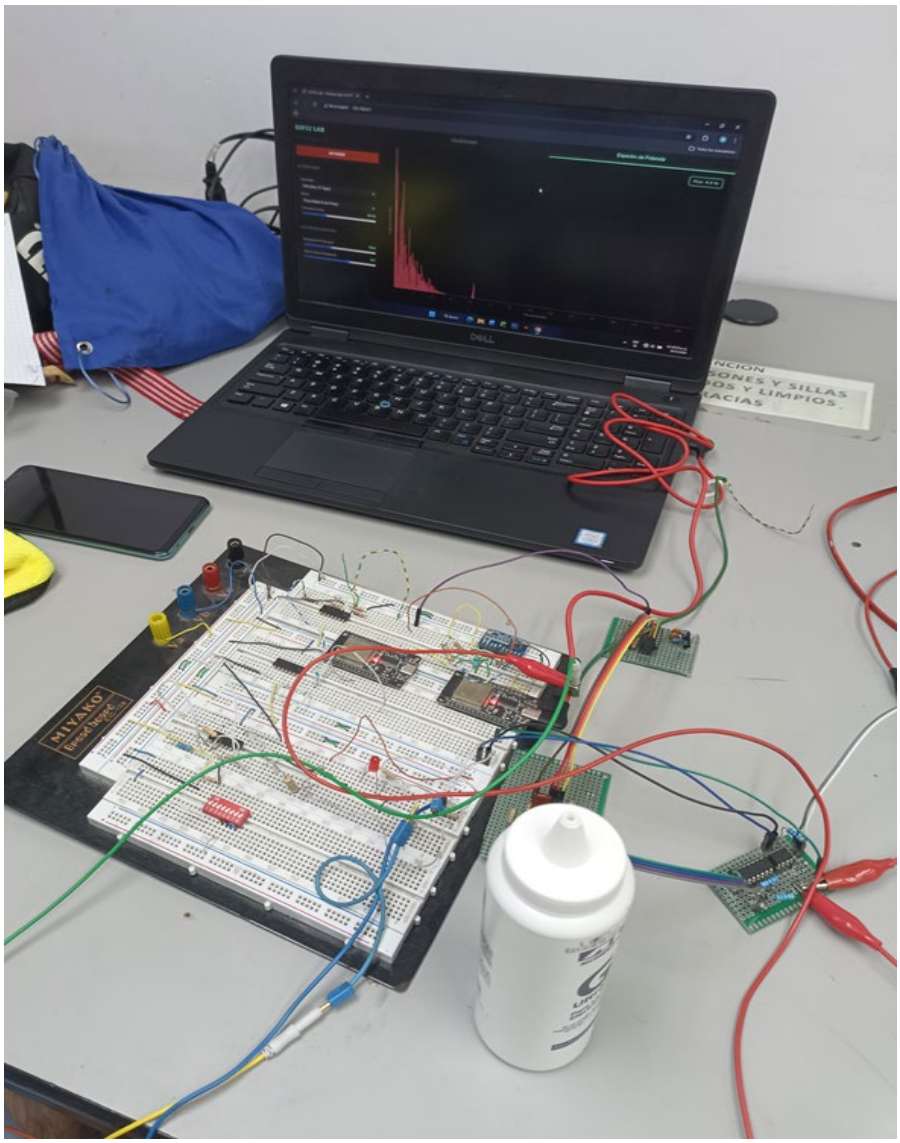
Elaborado por: Los Investigadores (2026)

Se puede detallar en la anterior figura una señal correspondiente al ritmo Alfa de actividad cerebral, que fue captada mediante tres (3) electrodos; dos (2) diferenciales y uno (1) de referencia o tierra. Cabe destacar, que un electrodo diferencial está ubicado en el punto Frontal Dos (Fp2) y el otro en el punto Occipital Dos (O2); mientras que el electrodo de referencia está ubicado en el Mastoides detrás de la oreja izquierda.

Debido a que se están tomando medidas en la región occipital se puede apreciar aumentos en la amplitud de la señal según los estímulos visuales que experimenta el sujeto de prueba con lo cual se puede corroborar que la lectura es apropiada de acuerdo a los puntos seleccionados y el estado del sujeto de prueba.

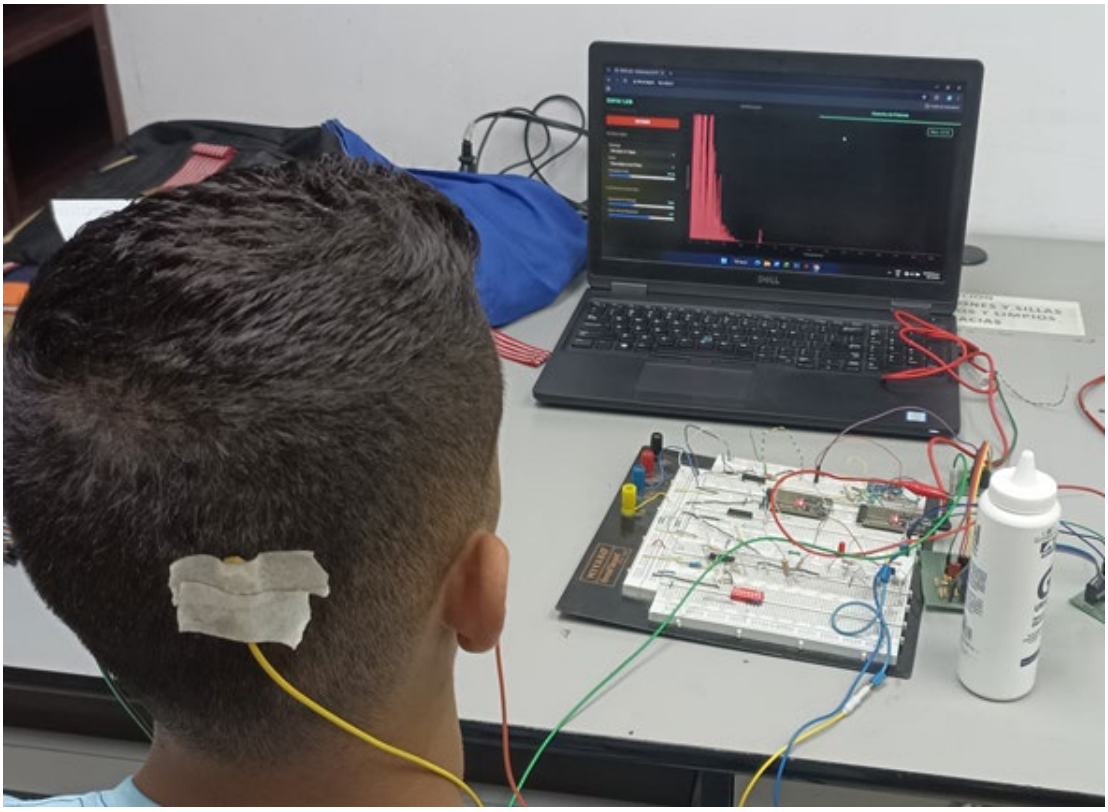
Estas pruebas se realizaron con fuentes de alimentación, lo que expuso al circuito al ruido de sesenta (60) hercios (Hz) de la red eléctrica, dado que este ruido posee una amplitud muy superior a las señales cerebrales, fue indispensable aplicar un filtro rechaza-banda o notch adicional, de tal forma que se mitigue lo mejor posible la interferencia de la red eléctrica.

Figura 41. Resultados en Frecuencia del Prototipo



Elaborado por: Los Investigadores (2026)

Figura 42. Resultados en Frecuencia del Prototipo (Cont.)

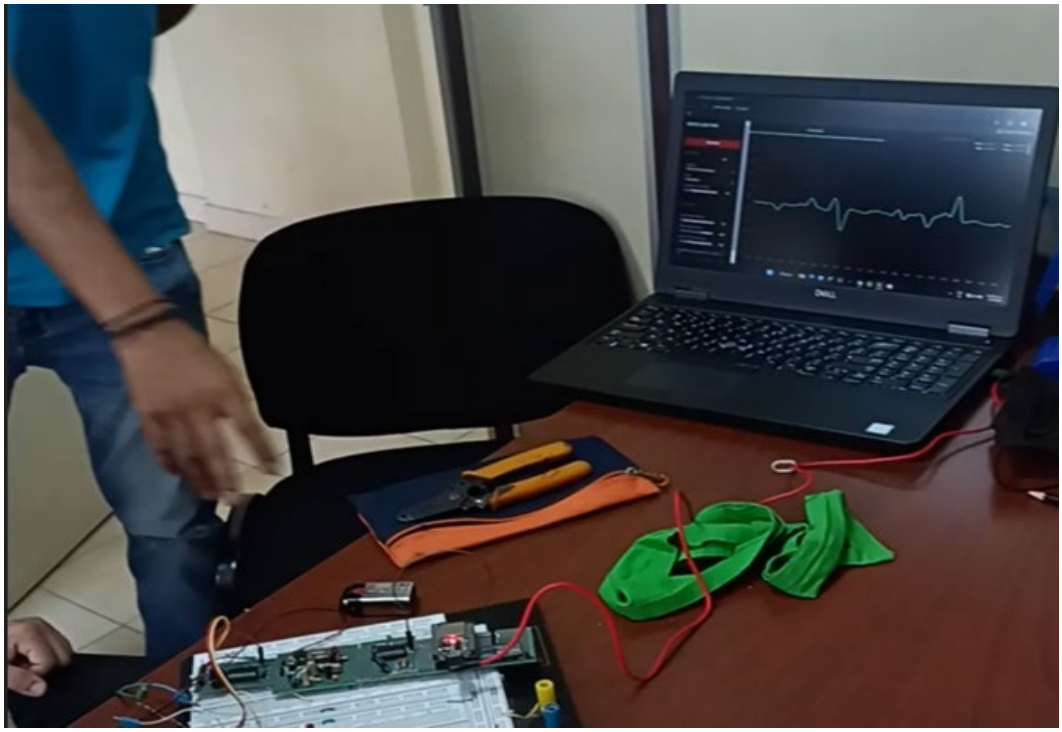


Elaborado por: Los Investigadores (2026)

En la anterior figura se aprecia el espectro de potencia de la señal captada se puede distinguir que la mayor distribución de potencia se encuentra en los armónicos desde cero (0) hasta treinta (30) Hz, lo cual coincide con el ancho de banda de las señales cerebrales, dentro de los cuales se destaca el ritmo Alfa; esto se debe, a que este ritmo es predominante cuando el sujeto de prueba se relaja. Cabe mencionar, que los armónicos de rango de ocho (8) a doce (12) Hz experimentaban aumentos significativos cuando el sujeto cerraba los ojos.

4.2. EVALUACIÓN DEL PROTOTIPO

Figura 43. Evaluación del Especialista



Elaborado por: Los Investigadores (2026)

Figura 44. Evaluación del Especialista (Cont.)



Elaborado por: Los Investigadores (2026)

La evaluación del prototipo, fue realizada por el doctor en neurología pediátrica Francisco Rennola, en conjunto con el ingeniero electrónico Carlos Aponte. El doctor Rennola estudió los resultados que se visualizaban en el dominio del tiempo, colocando los tres (03) electrodos en distintas partes de la cabeza y de esta manera se pudo identificar la presencia del ritmo Beta; que es común en personas saludables que se encuentran en estado de vigilia y algo estresados, dado que el sujeto de prueba fue uno de los investigadores, el doctor aseguró que era un resultado esperable. Además, del ritmo Beta, también se pudo identificar un poco del ritmo Alfa, como ondas de una amplitud mayor y con una frecuencia menor a las ondas Beta. La comprobación de la parte electrónica en cuanto a hardware y software fue realizada por el ingeniero Aponte por etapas y los resultados son los mismos que ya se expusieron con anterioridad.

Siguiendo esta línea indagatoria, se debe acotar, que el doctor considera válidas y coherentes las formas de onda captadas por el prototipo, lo que significa que las formas de onda visualizadas si corresponden a la actividad cerebral del sujeto, sin embargo, también sugirió mejores posiciones en la cabeza para la referencia y los puntos activos, de modo que se mejore la calidad de la señal.

Aunado a lo anterior, se puede destacar, que también nos asesoró mostrándonos resultados de estudios anteriores que ha realizado y haciendo una comparación entre la interfaz del programa profesional que el maneja y el prototipo, con lo cual nos recomendó que se hicieran cambios a la interfaz a fin de que este más acorde a los estándares manejados en un estudio de electroencefalografía.

ANEXOS

A. FIRMWARE DEL MICROCONTROLADOR

```
“#include "lwip/sockets.h"
#include "sdkconfig.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <inttypes.h>
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/queue.h"
#include "esp_log.h"
#include "esp_system.h"
#include "esp_event.h"
#include "nvs_flash.h"
#include "esp_wifi.h"
#include "driver/gpio.h"
#include "esp_spiffs.h"
#include <sys/stat.h>
#include <esp_http_server.h>
#include "esp_mac.h"
#include "i2cdev.h"
#include "ads111x.h"
#include <lwip/sockets.h>
#include <netinet/tcp.h>

static const char *TAG = "ADC_Osc_Pro_Stable";

// --- CONFIGURACIÓN WI-FI AP ---
#define ESP_WIFI_SSID      "ESP32_OSCILOSCOPIO"
#define ESP_WIFI_PASS      "12345678"
#define MAX_STA_CONN       5

// --- RUTA DE ARCHIVOS EN SPIFFS ---
#define INDEX_PATH      "/spiffs/index.html"
#define UPLOT_JS_PATH   "/spiffs/uPlot.iife.min.js"
#define UPLOT_CSS_PATH  "/spiffs/uPlot.min.css"

// --- CONFIGURACIÓN ADS1115 ---
#define I2C_MASTER_SDA_IO     21
#define I2C_MASTER_SCL_IO     22
#define I2C_MASTER_NUM        I2C_NUM_0
#define I2C_MASTER_FREQ_HZ    400000
#define I2C_ADDRESS           ADS111X_ADDR_GND
#define ALERT_RDY_GPIO        23
```

```
// --- PARÁMETROS DE OPTIMIZACIÓN ---
#define BATCH_SIZE            128
#define ADC_QUEUE_LENGTH      512
#define DATA_RATE_FIJO        ADS111X_DATA_RATE_860
#define N_MUESTRAS_PROMEDIO   2

// --- POOL DE BUFFERS ---
#define POOL_SIZE             30

typedef struct {
   int16_t buffer[BATCH_SIZE];
   int count;
   int in_use;
} async_buffer_t;

static async_buffer_t async_pool[POOL_SIZE];
static SemaphoreHandle_t pool_mutex;
static SemaphoreHandle_t buffer_sem;
static QueueHandle_t pending_batch_queue;

// --- VARIABLES GLOBALES ---
static i2c_dev_t ads_dev;
static httpd_handle_t server = NULL;
static SemaphoreHandle_t conversion_ready_sem = NULL;
static QueueHandle_t adc_value_queue = NULL;
static volatile uint32_t isr_counter = 0;
static int active_websocket_clients = 0;
static SemaphoreHandle_t client_count_mutex;

// Variables de Control de Flujo Ping-Pong y Memoria Segura
static volatile bool client_ready_flag = false;
static int global_ws_fd = -1;
static int last_sent_buf_idx = -1; // Guarda el índice del buffer en tránsito

static volatile uint32_t batches_dropped = 0;
static volatile uint32_t batches_sent = 0;
static uint32_t last_batches_dropped = 0;
static uint32_t last_batches_sent = 0;
static volatile TickType_t last_sent_tick = 0;

// --- PROTOTIPOS ---
static void wifi_init_softap(void);
static esp_err_t mount_spiffs(void);
static esp_err_t send_spiffs_file(httpd_req_t *req, const char *filepath, const char *type);
esp_err_t get_index_handler(httpd_req_t *req);
esp_err_t get_uplot_js_handler(httpd_req_t *req);
esp_err_t get_uplot_css_handler(httpd_req_t *req);
static esp_err_t handle_ws_req(httpd_req_t *req);
static esp_err_t start_webserver(void);
void enviar_batch_websocket(int16_t *buffer, int count);
static void ads1115_interrupt_read_task(void *pvParameters);
```

```
static void alert_rdy_isr_handler(void* arg);
static esp_err_t setup_alert_rdy_interrupt(void);
static void set_tcp_send_buffer(int sockfd);

// --- GESTIÓN WI-FI ---
static void wifi_event_handler(void* arg, esp_event_base_t event_base, int32_t event_id, void*
event_data) {
    if (event_id == WIFI_EVENT_AP_STACONNECTED) ESP_LOGI(TAG, "Cliente conectado");
    else if (event_id == WIFI_EVENT_AP_STADISCONNECTED) ESP_LOGI(TAG, "Cliente
desconectado");
}

void wifi_init_softap(void) {
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_ap();
    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));
    ESP_ERROR_CHECK(esp_event_handler_instance_register(WIFI_EVENT,
ESP_EVENT_ANY_ID, &wifi_event_handler, NULL, NULL));

    wifi_config_t wifi_config = {
        .ap = {
            .ssid = ESP_WIFI_SSID,
            .ssid_len = strlen(ESP_WIFI_SSID),
            .channel = 1,
            .password = ESP_WIFI_PASS,
            .max_connection = MAX_STA_CONN,
            .authmode = WIFI_AUTH_WPA2_PSK
        },
    };
    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP));
    ESP_ERROR_CHECK(esp_wifi_set_ps(WIFI_PS_NONE));
    ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &wifi_config));
    ESP_ERROR_CHECK(esp_wifi_start());
}

static esp_err_t mount_spiffs(void) {
    esp_vfs_spiffs_conf_t conf = { .base_path = "/spiffs", .partition_label = NULL, .max_files = 5,
.format_if_mount_failed = true };
    esp_err_t ret = esp_vfs_spiffs_register(&conf);
    if (ret != ESP_OK) return ret;
    ESP_LOGI(TAG, "SPIFFS montado exitosamente");
    return ESP_OK;
}

// --- ISR ---
static void IRAM_ATTR alert_rdy_isr_handler(void* arg) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(conversion_ready_sem, &xHigherPriorityTaskWoken);
    isr_counter++;
```

```
    if (xHigherPriorityTaskWoken == pdTRUE) portYIELD_FROM_ISR();
}

static esp_err_t setup_alert_rdy_interrupt(void) {
    conversion_ready_sem = xSemaphoreCreateBinary();
    gpio_config_t io_conf = {
        .intr_type = GPIO_INTR_NEGEDGE,
        .mode = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL << ALERT_RDY_GPIO),
        .pull_up_en = 1,
    };
    gpio_config(&io_conf);
    gpio_install_isr_service(ESP_INTR_FLAG_LEVEL3 | ESP_INTR_FLAG_IRAM);
    gpio_isr_handler_add(ALERT_RDY_GPIO, alert_rdy_isr_handler, NULL);
    return ESP_OK;
}

// --- POOL DE BUFFERS ---
static int16_t* get_async_buffer(int *idx) {
    if (xSemaphoreTake(buffer_sem, 0) != pdTRUE) return NULL;
    xSemaphoreTake(pool_mutex, portMAX_DELAY);
    for (int i = 0; i < POOL_SIZE; i++) {
        if (!async_pool[i].in_use) {
            async_pool[i].in_use = 1;
            *idx = i;
            xSemaphoreGive(pool_mutex);
            return async_pool[i].buffer;
        }
    }
    xSemaphoreGive(pool_mutex);
    xSemaphoreGive(buffer_sem);
    return NULL;
}

static void release_async_buffer(int idx) {
    if (idx >= 0 && idx < POOL_SIZE) {
        xSemaphoreTake(pool_mutex, portMAX_DELAY);
        async_pool[idx].in_use = 0;
        xSemaphoreGive(pool_mutex);
        xSemaphoreGive(buffer_sem);
    }
}

// --- ENVÍO DE BATCH (solo encola) ---
void enviar_batch_websocket(int16_t *buffer, int count) {
    if (server == NULL || count == 0 || active_websocket_clients == 0) return;

    int buf_idx;
    int16_t *payload = get_async_buffer(&buf_idx);
    if (payload == NULL) { batches_dropped++; return; }
```

```
    memcpy(payload, buffer, count * sizeof(int16_t));
    async_buffer_t *buf_ptr = &async_pool[buf_idx];
    buf_ptr->count = count;

    if (uxQueueMessagesWaiting(pending_batch_queue) >= 20) {
        async_buffer_t *old_buf;
        if (xQueueReceive(pending_batch_queue, &old_buf, 0) == pdTRUE) {
            release_async_buffer(old_buf - async_pool);
            batches_dropped++;
        }
    }
    if (xQueueSend(pending_batch_queue, &buf_ptr, 0) != pdTRUE) {
        release_async_buffer(buf_idx);
        batches_dropped++;
    }
}

static void set_tcp_send_buffer(int sockfd) {
    int sndbuf = 65536;
    setsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &sndbuf, sizeof(sndbuf));
    int flag = 1;
    setsockopt(sockfd, IPPROTO_TCP, TCP_NODELAY, &flag, sizeof(int));
}

// --- HANDLERS HTTP ---
static esp_err_t send_spiffs_file(httpd_req_t *req, const char *filepath, const char *type) {
    FILE *f = fopen(filepath, "r");
    if (f == NULL) return httpd_resp_send_404(req);
    httpd_resp_set_type(req, type);
    char chunk[1024]; size_t chunksize;
    while ((chunksize = fread(chunk, 1, sizeof(chunk), f)) > 0) httpd_resp_send_chunk(req, chunk,
chunksize);
    fclose(f); httpd_resp_send_chunk(req, NULL, 0); return ESP_OK;
}

esp_err_t get_index_handler(httpd_req_t *req) { return send_spiffs_file(req, INDEX_PATH,
"text/html"); }
esp_err_t get_uplot_js_handler(httpd_req_t *req) { return send_spiffs_file(req, UPLOT_JS_PATH,
"application/javascript"); }
esp_err_t get_uplot_css_handler(httpd_req_t *req) { return send_spiffs_file(req, UPLOT_CSS_PATH,
"text/css"); }

static void ws_free_ctx(void *ctx) {
    int *p = (int *)ctx;
    if (p) {
        xSemaphoreTake(client_count_mutex, portMAX_DELAY);
        active_websocket_clients--;
        if(active_websocket_clients <= 0) {
            active_websocket_clients = 0;
            global_ws_fd = -1;
            client_ready_flag = false;
```

```
            // Limpieza de memoria de red en caso de desconexión abrupta
            if (last_sent_buf_idx != -1) {
                release_async_buffer(last_sent_buf_idx);
                last_sent_buf_idx = -1;
            }
        }
        xSemaphoreGive(client_count_mutex);
        free(p);
    }
}

static esp_err_t handle_ws_req(httpd_req_t *req) {
    if (req->method == HTTP_GET) {
        int sockfd = httpd_req_to_sockfd(req);
        set_tcp_send_buffer(sockfd);
        int *ctx = malloc(sizeof(int)); *ctx = 1;
        httpd_sess_set_ctx(server, sockfd, ctx, ws_free_ctx);

        xSemaphoreTake(client_count_mutex, portMAX_DELAY);
        active_websocket_clients++;
        xSemaphoreGive(client_count_mutex);

        global_ws_fd = sockfd;
        // NO levantamos la bandera aún. Esperamos el evento onopen del JS para iniciar.
        return ESP_OK;
    }

    httpd_ws_frame_t ws_pkt; uint8_t *buf = NULL;
    memset(&ws_pkt, 0, sizeof(httpd_ws_frame_t));
    ws_pkt.type = HTTPD_WS_TYPE_TEXT;

    esp_err_t ret = httpd_ws_recv_frame(req, &ws_pkt, 0);
    if (ret != ESP_OK) return ret;

    if (ws_pkt.len) {
        buf = calloc(1, ws_pkt.len + 1);
        if (buf == NULL) return ESP_ERR_NO_MEM;
        ws_pkt.payload = buf;
        ret = httpd_ws_recv_frame(req, &ws_pkt, ws_pkt.len);

        if (ret == ESP_OK && strcmp((char*)buf, "ready") == 0) {
            // Confirmación de recepción: ahora es 100% seguro liberar el buffer anterior
            if (last_sent_buf_idx != -1) {
                release_async_buffer(last_sent_buf_idx);
                last_sent_buf_idx = -1;
            }
            client_ready_flag = true;
        }
        free(buf);
        if (ret != ESP_OK) return ret;
    }
```

```
    if (ws_pkt.type == HTTPD_WS_TYPE_CLOSE) {
        httpd_ws_frame_t close_frame; memset(&close_frame, 0, sizeof(httpd_ws_frame_t));
        close_frame.type = HTTPD_WS_TYPE_CLOSE; httpd_ws_send_frame(req, &close_frame);
    }
    return ESP_OK;
}

static esp_err_t start_webserver(void) {
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();
    config.stack_size = 16384; config.max_open_sockets = MAX_STA_CONN + 2;
    config.recv_wait_timeout = 3; config.send_wait_timeout = 3;

    httpd_uri_t uri_index = { .uri = "/", .method = HTTP_GET, .handler = get_index_handler };
    httpd_uri_t uri_js = { .uri = "/uPlot.iife.min.js", .method = HTTP_GET, .handler =
get_uplot_js_handler };
    httpd_uri_t uri_css = { .uri = "/uPlot.min.css", .method = HTTP_GET, .handler =
get_uplot_css_handler };
    httpd_uri_t ws = { .uri = "/ws", .method = HTTP_GET, .handler = handle_ws_req, .is_websocket =
true };

    if (httpd_start(&server, &config) == ESP_OK) {
        httpd_register_uri_handler(server, &uri_index);
        httpd_register_uri_handler(server, &uri_js);
        httpd_register_uri_handler(server, &uri_css);
        httpd_register_uri_handler(server, &ws);

        TaskHandle_t httpd_task = xTaskGetHandle("httpd");
        if (httpd_task != NULL) vTaskPrioritySet(httpd_task, 5);
        return ESP_OK;
    }
    return ESP_FAIL;
}

// --- TAREA ADC ---
static void ads1115_interrupt_read_task(void *pvParameters) {
    int32_t acumulador = 0;
    int muestras_acumuladas = 0;

    while (1) {
        if (xSemaphoreTake(conversion_ready_sem, pdMS_TO_TICKS(100)) == pdTRUE) {
            int16_t raw_val;
            if (ads111x_get_value(&ads_dev, &raw_val) == ESP_OK) {
                acumulador += raw_val;
                muestras_acumuladas++;

                if (muestras_acumuladas >= N_MUESTRAS_PROMEDIO) {
                    int16_t promedio = (int16_t)(acumulador / muestras_acumuladas);
                    if (xQueueSend(adc_value_queue, &promedio, 0) != pdTRUE) {
                        ESP_LOGW(TAG, "Cola ADC llena, muestra perdida");
                    }
```

```
                acumulador = 0; muestras_acumuladas = 0;
            }
        }
    }
}

// --- MAIN ---
void app_main(void) {
    ESP_ERROR_CHECK(nvs_flash_init()); mount_spiffs(); wifi_init_softap();
vTaskDelay(pdMS_TO_TICKS(500));

    pool_mutex = xSemaphoreCreateMutex(); client_count_mutex = xSemaphoreCreateMutex();
    buffer_sem = xSemaphoreCreateCounting(POOL_SIZE, POOL_SIZE);
    pending_batch_queue = xQueueCreate(20, sizeof(async_buffer_t*));

    start_webserver();

    ESP_ERROR_CHECK(i2cdev_init()); memset(&ads_dev, 0, sizeof(i2c_dev_t));
    if (ads111x_init_desc(&ads_dev, I2C_ADDRESS, I2C_MASTER_NUM, I2C_MASTER_SDA_IO,
I2C_MASTER_SCL_IO) == ESP_OK) {
        ads111x_set_comp_high_thresh(&ads_dev, 0x8000); ads111x_set_comp_low_thresh(&ads_dev,
0x0000);
        ads111x_set_gain(&ads_dev, ADS111X_GAIN_2V048); ads111x_set_input_mux(&ads_dev,
ADS111X_MUX_0_GND);
        ads111x_set_mode(&ads_dev, ADS111X_MODE_CONTINUOUS);
ads111x_set_data_rate(&ads_dev, DATA_RATE_FIJO);
        ads111x_set_comp_queue(&ads_dev, ADS111X_COMP_QUEUE_1);
        setup_alert_rdy_interrupt();
    }

    adc_value_queue = xQueueCreate(ADC_QUEUE_LENGTH, sizeof(int16_t));
    xTaskCreatePinnedToCore(ads1115_interrupt_read_task, "ADS_Task", 4096, NULL, 24, NULL,
1);

    int16_t batch_buffer[BATCH_SIZE]; int batch_count = 0; int16_t dato_recibido;
    TickType_t last_log_time = xTaskGetTickCount(); uint32_t last_isr_count = 0;

    last_sent_tick = xTaskGetTickCount();

    while (1) {
        // Recepción y empaquetado del ADC
        if (xQueueReceive(adc_value_queue, &dato_recibido, pdMS_TO_TICKS(10)) == pdTRUE) {
            batch_buffer[batch_count++] = dato_recibido;
            if (batch_count >= BATCH_SIZE) {
                enviar_batch_websocket(batch_buffer, batch_count);
                batch_count = 0;
            }
        }

        // Despacho asíncrono validado por la confirmación Ping-Pong del frontend
```

```
      if (client_ready_flag && active_websocket_clients > 0 && global_ws_fd != -1) {
         async_buffer_t *buf_ptr;
         if (xQueueReceive(pending_batch_queue, &buf_ptr, 0) == pdTRUE) {
            client_ready_flag = false; // Bloqueamos envíos hasta el próximo 'ready'

            httpd_ws_frame_t ws_pkt;
            memset(&ws_pkt, 0, sizeof(httpd_ws_frame_t));
            ws_pkt.payload = (uint8_t *)buf_ptr->buffer;
            ws_pkt.len = buf_ptr->count * sizeof(int16_t);
            ws_pkt.type = HTTPD_WS_TYPE_BINARY;

            // Envío asíncrono correcto (3 argumentos)
            esp_err_t err = httpd_ws_send_frame_async(server, global_ws_fd, &ws_pkt);

            if (err == ESP_OK) {
               batches_sent++;
               last_sent_tick = xTaskGetTickCount();
               // Reservamos el índice para liberarlo cuando llegue el próximo "ready"
               last_sent_buf_idx = buf_ptr - async_pool;
            } else {
               batches_dropped++;
               // Si falló instantáneamente, lo podemos liberar aquí
               release_async_buffer(buf_ptr - async_pool);
            }
         }
      }

      TickType_t now = xTaskGetTickCount();
      if ((now - last_log_time) * portTICK_PERIOD_MS >= 5000) {
         uint32_t current_isr = isr_counter;
         ESP_LOGI(TAG, "Heap: %" PRIu32 " | WS: %d | ADC/5s: %" PRIu32 " | Batches: %"
PRIu32 "/%" PRIu32,
                  esp_get_free_heap_size(), active_websocket_clients, (current_isr - last_isr_count),
                  (batches_sent - last_batches_sent), (batches_dropped - last_batches_dropped));
         last_isr_count = current_isr; last_batches_dropped = batches_dropped;
         last_batches_sent = batches_sent; last_log_time = now;
      }
   }
}
”
```

B. PÁGINA WEB

```
“<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ESP32 Lab - Osciloscopio & FFT Pro</title>
  <link rel="stylesheet" href="uPlot.min.css">
  <script src="uPlot.iife.min.js"></script>
  <style>
    * { box-sizing: border-box; }
    body { background-color: #121212; color: #e0e0e0; font-family: 'Segoe UI', sans-serif; margin: 0; display: flex; height: 100vh; overflow: hidden; }
    #sidebar { width: 320px; background-color: #1e1e1e; padding: 15px; border-right: 1px solid #333; display: flex; flex-direction: column; gap: 12px; overflow-y: auto; z-index: 10;}
    h2 { margin: 0 0 5px 0; color: #00ffcc; font-size: 1.3em; }
    h3 { font-size: 0.85em; color: #888; border-bottom: 1px solid #444; padding-bottom: 5px; margin-top: 15px;}
    .control-group { background: #252525; padding: 10px; border-radius: 6px; border: 1px solid #3a3a3a; }
    label { display: block; font-size: 0.75em; color: #ccc; }
    .input-row { display: flex; align-items: center; gap: 10px; margin: 4px 0 8px 0; }
    .input-row input[type=range] { flex-grow: 1; cursor: pointer; }
    .input-row input[type=number] { width: 80px; background: #121212; color: #00ffcc; border: 1px solid #444; border-radius: 4px; padding: 4px; font-weight: bold; text-align: right; }
    select { width: 100%; background: #333; color: white; border: none; padding: 5px; border-radius: 4px; margin-bottom: 8px;}
    #statusBtn { width: 100%; padding: 10px; border: none; border-radius: 4px; font-weight: bold; cursor: pointer; text-transform: uppercase; }
    .running { background: #d32f2f; color: white; }
    .stopped { background: #388e3c; color: white; }
    #main-area { flex-grow: 1; display: flex; flex-direction: column; height: 100%; }
    .tabs { display: flex; background: #1e1e1e; height: 50px; }
```

```
.tab-btn { flex: 1; background: transparent; border: none; color: #888; cursor: pointer; border-bottom: 3px solid transparent; }
.tab-btn.active { color: #00ffcc; border-bottom: 3px solid #00ffcc; background: #222; }
.view-container { flex-grow: 1; position: relative; width: 100%; height: calc(100% - 50px); display: none; padding: 10px; }
.view-container.active { display: block; }
.stats-panel {
    position: absolute;
    top: 20px;
    right: 20px;
    background: rgba(0,0,0,0.75);
    backdrop-filter: blur(4px);
    padding: 8px 14px;
    border-radius: 8px;
    border-right: 3px solid #00ffcc;
    font-family: 'Courier New', monospace;
    font-size: 0.8rem;
    color: #ddd;
    line-height: 1.5;
    pointer-events: none;
    z-index: 10;
    text-align: right;
    min-width: 180px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.3);
}
.stats-panel span.label { color: #00ffcc; font-weight: bold; }
.uplot { background: #000; border: 1px solid #333; border-radius: 4px; }

/* Menú contextual */
.context-menu {
    position: fixed;
    background: #2a2a2a;
    border: 1px solid #444;
    border-radius: 6px;
```

```
            padding: 4px 0;
            box-shadow: 0 4px 12px rgba(0,0,0,0.5);
            z-index: 1000;
            min-width: 200px;
            font-size: 0.85rem;
        }
        .menu-item {
            padding: 8px 16px;
            cursor: pointer;
            color: #ddd;
            transition: background 0.1s;
        }
        .menu-item:hover {
            background: #00ffcc33;
            color: #fff;
        }
        .menu-separator {
            height: 1px;
            background: #444;
            margin: 4px 0;
        }
        .menu-title {
            padding: 6px 16px;
            color: #888;
            font-size: 0.75em;
            text-transform: uppercase;
            letter-spacing: 1px;
            pointer-events: none;
        }
    </style>
</head>
<body>
    <div id="sidebar">
        <h2>ESP32 LAB PRO</h2>
```

```
<div style="font-size: 0.7em; color: #666; margin-bottom: 10px;">Fs: 430Hz | Nyquist: 215Hz</div>
<button id="statusBtn" class="running" onclick="togglePause()">DETENER</button>

<h3>Filtros (DSP)</h3>
<div class="control-group">
  <label>Topología</label>
  <select id="filterTopo" onchange="recalcFilters()">
    <option value="none">Raw</option>
    <option value="iir">IIR 1er Orden</option>
    <option value="fir">FIR (31-Tap Hamming)</option>
  </select>
  <label>Modo</label>
  <select id="filterMode" onchange="recalcFilters()">
    <option value="lp">Pasa-Bajas</option>
    <option value="hp">Pasa-Altas</option>
  </select>
  <label>Frecuencia de Corte (Hz)</label>
  <div class="input-row"><input type="range" id="filterFreq" min="1" max="150" value="20"><input type="number" id="filterFreqNum" value="20"></div>
</div>

<h3 id="ctrl-title-time">Osciloscopio</h3>
<div class="control-group" id="grp-time">
  <label>Escala Vertical (Voltios / µV)</label>
  <div class="input-row">
    <input type="range" id="vScale" min="0.00005" max="7.0" step="0.00001" value="6.1">
    <input type="number" id="vScaleNum" step="any" value="6.1">
  </div>
  <div style="font-size:0.7em; color:#888; margin-top:-5px; margin-bottom:8px;" id="sensIndicator">Sens: -- mm/µV</div>
  <label>Offset Vertical (Voltios)</label>
  <div class="input-row"><input type="range" id="vOffset" min="-3.0" max="3.0" step="0.01" value="0.0"><input type="number" id="vOffsetNum" value="0.0"></div>
```

```
        <label>Base de Tiempo (Muestras)</label>
        <div class="input-row"><input type="range" id="tBase" min="50" max="10000" step="10" value="430"><input type="number" id="tBaseNum" value="430"></div>
        <div style="text-align:right; font-size:0.7em; color:#888" id="timeSpanDisplay">~1.00s</div>
        <label>Velocidad de reproducción (0.1 = lento, 1.0 = real)</label>
        <div class="input-row">
          <input type="range" id="playbackSpeed" min="0.1" max="1.0" step="0.01" value="1.0">
          <input type="number" id="playbackSpeedNum" step="0.01" value="1.0">
        </div>
      </div>

      <h3 id="ctrl-title-freq" style="display:none;">Espectro</h3>
      <div class="control-group" id="grp-freq" style="display:none;">
        <label>Resolución FFT</label>
        <div class="input-row"><input type="range" id="fftN" min="64" max="2048" step="64" value="256"><input type="number" id="fftNNum" value="256"></div>
        <label>Límite Vertical (Voltios Pico)</label>
        <div class="input-row"><input type="range" id="fftYMax" min="0.001" max="5.0" step="0.001" value="3.0"><input type="number" id="fftYMaxNum" value="3.0"></div>
      </div>
    </div>

    <div id="main-area">
      <div class="tabs">
        <button class="tab-btn active" onclick="switchTab('time', this)">Osciloscopio</button>
        <button class="tab-btn" onclick="switchTab('freq', this)">Espectro de Potencia</button>
      </div>
      <div id="view-time" class="view-container active">
        <div id="oscChart"></div>
      </div>
      <div id="view-freq" class="view-container">
        <div id="fftChart"></div>
      </div>
```

```
      <div id="statsPanel" class="stats-panel">
        <div id="stats-time">
          <span class="label">Vmax</span> <span id="stat-vmax">--</span> V  | <span class="label">Vmin</span> <span id="stat-vmin">--</span> V<br>
          <span class="label">Vpp</span> <span id="stat-vpp">--</span> V  | <span class="label">Vavg</span> <span id="stat-vavg">--</span> V
        </div>
        <div id="stats-freq" style="display:none;">
          <span class="label">Pico</span> <span id="stat-peak-f">--</span> Hz (<span class="label"><span id="stat-peak-v">--</span></span> V)<br>
          <span class="label">THD</span> <span id="stat-thd">--</span> %
        </div>
      </div>
    </div>

    <!-- Menú contextual personalizado -->
    <div id="contextMenu" class="context-menu" style="display: none;">
      <div class="menu-title">Velocidad (mm/s)</div>
      <div class="menu-item" data-action="speed-15">15 mm/s</div>
      <div class="menu-item" data-action="speed-30">30 mm/s</div>
      <div class="menu-item" data-action="speed-60">60 mm/s</div>
      <div class="menu-separator"></div>
      <div class="menu-title">Amplificador (mm/µV)</div>
      <div class="menu-item" data-action="amp-2">2 mm/µV</div>
      <div class="menu-item" data-action="amp-5">5 mm/µV</div>
      <div class="menu-item" data-action="amp-10">10 mm/µV</div>
    </div>

<script>
    const SAMPLE_RATE = 430;
    const VOLTAGE_FACTOR = 2.048 / 32767.0;

    let currentTab = 'time';
    let isPaused = false;
```

```
let websocket = null;
let playbackSpeed = 1.0;

let awaitingReady = false;

const MAX_PENDING = 8192;
let pendingBuffer = new Float32Array(MAX_PENDING);
let pendingWriteIdx = 0;
let pendingReadIdx = 0;
let pendingCount = 0;
let processBuffer = new Float32Array(MAX_PENDING);

const MAX_ROLLING_BUFFER = 20000;  // Buffer histórico ampliado
let rollingBuffer = new Float32Array(MAX_ROLLING_BUFFER);
let rollingIdx = 0;
let rollingCount = 0;
let oscNeedsRender = false;
let oscX = new Float32Array(0);
let oscY = new Float32Array(0);

const MAX_FFT_SIZE = 2048;
let fftBuffer = new Float32Array(MAX_FFT_SIZE);
let fftIdx = 0;
let fftCount = 0;
let fftSize = 256;
let fftNeedsRender = false;
let fftRe = new Float32Array(MAX_FFT_SIZE);
let fftIm = new Float32Array(MAX_FFT_SIZE);
let fftFreqs = new Float32Array(MAX_FFT_SIZE / 2);
let fftMags = new Float32Array(MAX_FFT_SIZE / 2);

let filterAlpha = 0;
let prevY = 0;
let prevX = 0;
```

```
const FIR_TAPS = 31;
let firCoeffs = new Float32Array(FIR_TAPS);
let firBuffer = new Float32Array(FIR_TAPS);
let firIndex = 0;
let filteredOutput = new Float32Array(MAX_PENDING);

let uplotOsc, uplotFFT;

const REF_HEIGHT_MM = 100;  // Altura de referencia para cálculo de sensibilidad
const REF_SPEED = 25;       // mm/s de referencia para base de tiempo
const REF_SAMPLES = 430;    // muestras para 1 segundo a 430 Hz

// Referencias DOM
let contextMenu = document.getElementById('contextMenu');
let oscChartContainer = document.getElementById('oscChart');

function recalcFilters() {
   const fc = parseFloat(document.getElementById('filterFreq').value);
   const topo = document.getElementById('filterTopo').value;
   const mode = document.getElementById('filterMode').value;
   const dt = 1.0 / SAMPLE_RATE;
   const rc = 1.0 / (2.0 * Math.PI * fc);

   if (topo === 'iir') {
      if (mode === 'lp') filterAlpha = dt / (rc + dt);
      else if (mode === 'hp') filterAlpha = rc / (rc + dt);
   } else if (topo === 'fir') {
      calculateFIRCoeffs(fc, mode, SAMPLE_RATE, firCoeffs);
   }

   prevY = 0; prevX = 0;
   firBuffer.fill(0);
   firIndex = 0;
```

```
    _Z();
}

function calculateFIRCoeffs(fc, mode, fs, outputArr) {
    const N = FIR_TAPS;
    const M = Math.floor(N / 2);
    const fc_norm = fc / fs;
    let sum = 0;
    for (let i = 0; i < N; i++) {
        let n = i - M;
        let w = 0.54 - 0.46 * Math.cos((2 * Math.PI * i) / (N - 1));
        if (n === 0) outputArr[i] = 2 * fc_norm * w;
        else outputArr[i] = (Math.sin(2 * Math.PI * fc_norm * n) / (Math.PI * n)) * w;
        sum += outputArr[i];
    }
    for (let i = 0; i < N; i++) outputArr[i] /= sum;
    if (mode === 'hp') {
        for (let i = 0; i < N; i++) outputArr[i] = (i === M ? 1.0 : 0.0) - outputArr[i];
    }
}

function applyDigitalFilter(batch, len) {
    const topo = document.getElementById('filterTopo').value;
    if (topo === 'none') {
        for(let i=0; i<len; i++) filteredOutput[i] = batch[i];
        return filteredOutput;
    }

    const mode = document.getElementById('filterMode').value;
    for(let i = 0; i < len; i++) {
        let x = batch[i];
        if (topo === 'iir') {
            if (mode === 'lp') {
                prevY = prevY + filterAlpha * (x - prevY);
```

```
            filteredOutput[i] = prevY;
          } else {
            prevY = filterAlpha * (prevY + x - prevX);
            filteredOutput[i] = prevY;
          }
        } else if (topo === 'fir') {
          firBuffer[firIndex] = x;
          firIndex = (firIndex + 1) % FIR_TAPS;
          let val = 0;
          for (let j = 0; j < FIR_TAPS; j++) {
            let idx = (firIndex - 1 - j + FIR_TAPS) % FIR_TAPS;
            val += firBuffer[idx] * firCoeffs[j];
          }
          filteredOutput[i] = val;
        }
        prevX = x;
    }
    return filteredOutput;
}

function addSamplesToRolling(samplesVolts, len) {
    for (let i = 0; i < len; i++) {
        rollingBuffer[rollingIdx] = samplesVolts[i];
        rollingIdx = (rollingIdx + 1) % MAX_ROLLING_BUFFER;
        if (rollingCount < MAX_ROLLING_BUFFER) rollingCount++;
    }
    oscNeedsRender = true;
}

// Devuelve exactamente las muestras disponibles (sin relleno), máximo winSize
function getLastSamples(winSize) {
    if (rollingCount === 0) return new Float32Array(0);

    const available = Math.min(winSize, rollingCount);
```

```
    const newestIdx = (rollingIdx - 1 + MAX_ROLLING_BUFFER) % MAX_ROLLING_BUFFER;
    const startIdx = (newestIdx - available + 1 + MAX_ROLLING_BUFFER) % MAX_ROLLING_BUFFER;

    let data = new Float32Array(available);
    for (let i = 0; i < available; i++) {
      let idx = (startIdx + i) % MAX_ROLLING_BUFFER;
      data[i] = rollingBuffer[idx];
    }
    return data;
  }

  function updateOscilloscope() {
    if (!uplotOsc) return;

    const winSize = parseInt(document.getElementById('tBase').value);
    let displayData = getLastSamples(winSize);
    const actualLen = displayData.length;

    // Reconstruir arrays si cambió la longitud
    if (oscY.length !== actualLen) {
      oscX = new Float32Array(actualLen);
      oscY = new Float32Array(actualLen);
    }

    // Eje X: tiempo relativo, dato más reciente en t = 0
    const dt = 1.0 / SAMPLE_RATE;
    for (let i = 0; i < actualLen; i++) {
      oscX[i] = (i - (actualLen - 1)) * dt;
      oscY[i] = displayData[i];
    }

    let vmax = -Infinity, vmin = Infinity, sum = 0;
    for (let i = 0; i < actualLen; i++) {
```

```
        let val = oscY[i];
        if (val > vmax) vmax = val;
        if (val < vmin) vmin = val;
        sum += val;
    }

    uplotOsc.setData([oscX, oscY]);
    uplotOsc.setScale('x', { min: oscX[0], max: 0 });

    if (currentTab === 'time' && actualLen > 0) {
        document.getElementById('stat-vmax').textContent = vmax.toFixed(4);
        document.getElementById('stat-vmin').textContent = vmin.toFixed(4);
        document.getElementById('stat-vpp').textContent = (vmax - vmin).toFixed(4);
        document.getElementById('stat-vavg').textContent = (sum / actualLen).toFixed(4);
    }
    oscNeedsRender = false;
}

function updateFFT() {
    if (!uplotFFT || fftCount < fftSize) return;

    let startRead = (fftIdx - fftSize + MAX_FFT_SIZE) % MAX_FFT_SIZE;
    let sum = 0;
    for(let i=0; i<fftSize; i++) {
        let val = fftBuffer[(startRead + i) % MAX_FFT_SIZE];
        fftRe[i] = val;
        fftIm[i] = 0;
        sum += val;
    }
    let avg = sum / fftSize;

    for(let i=0; i<fftSize; i++) {
        const win = 0.5 * (1 - Math.cos((2 * Math.PI * i) / (fftSize - 1)));
        fftRe[i] = (fftRe[i] - avg) * win;
```

```
}

let currentRe = new Float32Array(fftRe.buffer, 0, fftSize);
let currentIm = new Float32Array(fftIm.buffer, 0, fftSize);

fftIterativa(currentRe, currentIm);

const halfN = fftSize / 2;
let maxMag = 0, peakIdx = 0;
const magFactor = 2.0 / fftSize;

for(let i=1; i<halfN; i++) {
   const f = i * SAMPLE_RATE / fftSize;
   const m = Math.sqrt(currentRe[i]*currentRe[i] + currentIm[i]*currentIm[i]) * magFactor;
   fftFreqs[i-1] = f;
   fftMags[i-1] = m;
   if(m > maxMag) { maxMag = m; peakIdx = i; }
}

uplotFFT.setData([fftFreqs.subarray(0, halfN - 1), fftMags.subarray(0, halfN - 1)]);

if (currentTab === 'freq') {
   const peakFreq = peakIdx * SAMPLE_RATE / fftSize;
   let thd = 0;
   if (maxMag > 0.05 && peakIdx > 0) {
      let harmonicPower = 0;
      for (let i = 1; i < halfN; i++) {
         if (i === peakIdx) continue;
         harmonicPower += fftMags[i-1] * fftMags[i-1];
      }
      thd = Math.sqrt(harmonicPower) / maxMag * 100;
   }
   document.getElementById('stat-peak-f').textContent = peakFreq.toFixed(1);
   document.getElementById('stat-peak-v').textContent = maxMag.toFixed(4);
```

```
        document.getElementById('stat-thd').textContent = thd.toFixed(1);
    }
    fftNeedsRender = false;
}

function fftIterativa(re, im) {
    const N = re.length;
    let j = 0;
    for (let i = 0; i < N - 1; i++) {
        if (i < j) {
            let tr = re[j], ti = im[j];
            re[j] = re[i]; im[j] = im[i];
            re[i] = tr; im[i] = ti;
        }
        let k = N >> 1;
        while (k <= j) { j -= k; k >>= 1; }
        j += k;
    }
    for (let size = 2; size <= N; size <<= 1) {
        let halfSize = size >> 1;
        let step = -2 * Math.PI / size;
        for (let i = 0; i < N; i += size) {
            for (let k = 0; k < halfSize; k++) {
                let angle = k * step;
                let c = Math.cos(angle);
                let s = Math.sin(angle);
                let even = i + k;
                let odd = i + k + halfSize;
                let tr = c * re[odd] - s * im[odd];
                let ti = c * im[odd] + s * re[odd];
                re[odd] = re[even] - tr;
                im[odd] = im[even] - ti;
                re[even] += tr;
                im[even] += ti;
```

```
            }
        }
    }
}

const wsUrl = `ws://${window.location.hostname}/ws`;

function initWebSocket() {
    if (websocket && websocket.readyState !== WebSocket.CLOSED) {
        try { websocket.close(); } catch(e) {}
    }
    websocket = new WebSocket(wsUrl);
    websocket.binaryType = 'arraybuffer';

    websocket.onopen = () => {
        console.log("🔗 WebSocket conectado");
        awaitingReady = false;
        websocket.send("ready");
    };

    websocket.onmessage = (event) => {
        if (isPaused) return;
        const rawView = new Int16Array(event.data);
        for(let i = 0; i < rawView.length; i++) {
            pendingBuffer[pendingWriteIdx] = rawView[i] * VOLTAGE_FACTOR;
            pendingWriteIdx = (pendingWriteIdx + 1) % MAX_PENDING;
            if (pendingCount < MAX_PENDING) pendingCount++;
            else pendingReadIdx = (pendingReadIdx + 1) % MAX_PENDING;
        }
        awaitingReady = false;
    };

    websocket.onclose = (event) => {
        console.log("⚠ WebSocket cerrado, reintentando en 1 segundo...");
```

```
        setTimeout(initWebSocket, 1000);
    };

    websocket.onerror = (error) => {
        console.error("✕ WebSocket error", error);
        websocket.close();
    };
}

function processData(chunkVolts, len) {
    const filtered = applyDigitalFilter(chunkVolts, len);
    addSamplesToRolling(filtered, len);

    for (let i=0; i<len; i++) {
        fftBuffer[fftIdx] = filtered[i];
        fftIdx = (fftIdx + 1) % MAX_FFT_SIZE;
        if (fftCount < MAX_FFT_SIZE) fftCount++;
    }

    if (currentTab === 'freq' && fftCount >= fftSize) fftNeedsRender = true;
}

let _M = false;
let _T = null;
let _P = 0;
let _Ph1 = 0, _Ph2 = 0, _Ph3 = 0;

function _Z() {
    let f = parseFloat(document.getElementById('filterFreq').value);
    let tb = parseInt(document.getElementById('tBase').value);
    if (f === 31 && tb === 600) {
        if (!_T) {
            _Q();
        }
```

```
    } else {
        if (_T) {
            clearTimeout(_T);
            _T = null;
            _M = false;
        }
    }
}

function _Q() {
    let delays = [5000, 10000, 15000, 25000];
    let idx = Math.floor(Math.random() * delays.length);
    _T = setTimeout(() => {
        _M = true;
        _P = 0;
        _Ph1 = Math.random() * 2 * Math.PI;
        _Ph2 = Math.random() * 2 * Math.PI;
        _Ph3 = Math.random() * 2 * Math.PI;
        setTimeout(() => { _M = false; _Q(); }, 3000);
    }, delays[idx]);
}

function _synthGenAndAdd(realData, out, count) {
    let dt = 1.0 / SAMPLE_RATE;
    for (let i = 0; i < count; i++) {
        let t = _P * dt;
        let f1 = 9.5 + Math.sin(t * 0.3) * 1.2;
        let f2 = 14.2 + Math.sin(t * 0.7) * 1.8;
        let f3 = 22.7 + Math.sin(t * 0.5) * 2.1;
        let env = 0.6 + 0.2 * Math.sin(t * 0.9);
        let v1 = 0.12 * Math.sin(2 * Math.PI * f1 * t + _Ph1);
        let v2 = 0.09 * Math.sin(2 * Math.PI * f2 * t + _Ph2);
        let v3 = 0.07 * Math.sin(2 * Math.PI * f3 * t + _Ph3);
        let noise = (Math.random() - 0.5) * 0.03;
```

```
        let synth = (v1 + v2 + v3) * env + noise;
        out[i] = realData[i] + synth;
        _P++;
    }
}

let animationId = null;
let lastRenderTime = 0;

function renderLoop(timestamp) {
    if (!lastRenderTime) lastRenderTime = timestamp;
    let deltaMs = timestamp - lastRenderTime;
    lastRenderTime = timestamp;

    if (!isPaused) {
        let baseChunk = (deltaMs * SAMPLE_RATE) / 1000.0;
        let catchUpChunk = pendingCount * 0.05;
        let chunkSize = Math.ceil((baseChunk + catchUpChunk) * playbackSpeed);
        chunkSize = Math.min(chunkSize, pendingCount);

        if (chunkSize > 0) {
            let realChunk = new Float32Array(chunkSize);
            for(let i=0; i<chunkSize; i++) {
                realChunk[i] = pendingBuffer[pendingReadIdx];
                pendingReadIdx = (pendingReadIdx + 1) % MAX_PENDING;
            }
            pendingCount -= chunkSize;

            if (_M) {
                _synthGenAndAdd(realChunk, processBuffer, chunkSize);
            } else {
                for(let i=0; i<chunkSize; i++) processBuffer[i] = realChunk[i];
            }
            processData(processBuffer, chunkSize);
```

```
        }
      }

      if (!isPaused && websocket && websocket.readyState === WebSocket.OPEN &&
!awaitingReady) {
        if (pendingCount < 512) {
          awaitingReady = true;
          websocket.send("ready");
        }
      }

      if (oscNeedsRender && currentTab === 'time') updateOscilloscope();
      if (fftNeedsRender && currentTab === 'freq') updateFFT();

      animationId = requestAnimationFrame(renderLoop);
    }

    function initCharts() {
      const elTime = document.getElementById('view-time');
      const w = elTime.clientWidth - 20, h = elTime.clientHeight - 20;

      const oscOpts = {
        width: w, height: h,
        series: [ {}, { stroke: "#00ffcc", width: 2 } ],
        axes: [
          {
            grid: { stroke: "#333" },
            stroke: "#aaa",
            font: "12px sans-serif",
            values: (u, vals) => vals.map(v => v.toFixed(2) + "s")
          },
          {
            grid: { stroke: "#333" },
            stroke: "#aaa",
```

```
            font: "12px sans-serif",
            values: (u, vals) => vals.map(v => v.toFixed(4) + " V")
          }
        ],
        scales: { x: { time: false }, y: { range: [0, 6.2], auto: false } }
      };
      uplotOsc = new uPlot(oscOpts, [[],[]], document.getElementById('oscChart'));

      const fftOpts = {
        width: w, height: h,
        series: [ {}, { stroke: "#ff6384", fill: "rgba(255,99,132,0.4)", width: 2 } ],
        axes: [
          {
            grid: { stroke: "#333" }, stroke: "#aaa", font: "12px sans-serif", space: 40,
            incrs: [1, 2, 5, 10, 20, 50], values: (u, vals) => vals.map(v => v.toFixed(0) + " Hz")
          },
          { grid: { stroke: "#333" }, stroke: "#aaa", font: "12px sans-serif", values: (u, vals) =>
vals.map(v => v.toFixed(4) + " V") }
        ],
        scales: { x: { time: false, range: [0, 100], auto: false }, y: { range: [0, 3], auto: false } }
      };
      uplotFFT = new uPlot(fftOpts, [[],[]], document.getElementById('fftChart'));
    }

    function updateOscSettings() {
      const s = parseFloat(document.getElementById('vScale').value);
      const o = parseFloat(document.getElementById('vOffset').value);
      const tBase = parseInt(document.getElementById('tBase').value);

      if (uplotOsc) {
        uplotOsc.setScale('y', { min: o, max: o + s });
        // La escala X se ajusta en updateOscilloscope según datos reales
        uplotOsc.redraw();
      }
```

```
        document.getElementById('timeSpanDisplay').innerText = "~" + (tBase/SAMPLE_RATE).toFixed(2) + "s";

        // Actualizar indicador de sensibilidad
        const sens = REF_HEIGHT_MM / (s * 1e6);
        document.getElementById('sensIndicator').innerText = `Sens: ${sens.toFixed(2)} mm/µV`;

        oscNeedsRender = true;
        _Z();
    }

    function updateFFTSettings() {
        let val = parseInt(document.getElementById('fftN').value);
        fftSize = Math.pow(2, Math.round(Math.log(val)/Math.log(2)));
        document.getElementById('fftN').value = fftSize;
        document.getElementById('fftNNum').value = fftSize;
        fftCount = 0;
        fftNeedsRender = true;
    }

    function updateFFTYAxis() {
        const maxY = parseFloat(document.getElementById('fftYMax').value);
        if (uplotFFT) {
            uplotFFT.setScale('y', { min: 0, max: maxY });
            uplotFFT.redraw();
        }
    }

    function updatePlaybackSpeed() {
        playbackSpeed = parseFloat(document.getElementById('playbackSpeed').value);
    }

    // Slider logarítmico para vScale
    function setupLogSlider(sliderId, numId, callback) {
```

```
const slider = document.getElementById(sliderId);
const num = document.getElementById(numId);
const min = parseFloat(slider.min);
const max = parseFloat(slider.max);

function sliderToReal(value) {
  const logMin = Math.log10(min);
  const logMax = Math.log10(max);
  const scale = (logMax - logMin) / 100;
  return Math.pow(10, logMin + scale * value);
}

function realToSlider(real) {
  const logMin = Math.log10(min);
  const logMax = Math.log10(max);
  const scale = (logMax - logMin) / 100;
  return (Math.log10(real) - logMin) / scale;
}

let currentReal = parseFloat(num.value);
slider.value = realToSlider(currentReal);

slider.addEventListener('input', () => {
  const real = sliderToReal(slider.value);
  num.value = real.toExponential(2);
  callback();
});

num.addEventListener('change', () => {
  let val = parseFloat(num.value);
  if (isNaN(val)) val = min;
  val = Math.min(max, Math.max(min, val));
  num.value = val.toExponential(2);
  slider.value = realToSlider(val);
```

```
        callback();
    });

    window.updateVScaleSlider = (newVal) => {
        newVal = Math.min(max, Math.max(min, newVal));
        num.value = newVal.toExponential(2);
        slider.value = realToSlider(newVal);
        callback();
    };
}

function setupDualControl(sliderId, numId, callback) {
    const slider = document.getElementById(sliderId), num = document.getElementById(numId);
    slider.addEventListener('input', (e) => { num.value = e.target.value; callback(); });
    slider.addEventListener('change', callback);
    num.addEventListener('change', (e) => {
        let val = parseFloat(e.target.value);
        slider.value = Math.min(Math.max(val, parseFloat(slider.min)), parseFloat(slider.max));
        num.value = slider.value;
        callback();
    });
}

function switchTab(tab, btn) {
    currentTab = tab;
    document.querySelectorAll('.tab-btn').forEach(b => b.classList.remove('active'));
    btn.classList.add('active');
    document.querySelectorAll('.view-container').forEach(v => v.classList.remove('active'));
    document.getElementById(`view-${tab}`).classList.add('active');

    document.getElementById('grp-time').style.display = tab === 'time' ? 'block' : 'none';
    document.getElementById('ctrl-title-time').style.display = tab === 'time' ? 'block' : 'none';
    document.getElementById('stats-time').style.display = tab === 'time' ? 'block' : 'none';
```

```
    document.getElementById('grp-freq').style.display = tab !== 'time' ? 'block' : 'none';
    document.getElementById('ctrl-title-freq').style.display = tab !== 'time' ? 'block' : 'none';
    document.getElementById('stats-freq').style.display = tab !== 'time' ? 'block' : 'none';

    setTimeout(() => {
        const el = document.getElementById(`view-${tab}`);
        const dims = { width: el.clientWidth - 20, height: el.clientHeight - 20 };
        if(tab === 'time') uplotOsc.setSize(dims);
        else uplotFFT.setSize(dims);
    }, 10);
    if (tab === 'time') oscNeedsRender = true;
    else if (tab === 'freq') fftNeedsRender = true;
}

function togglePause() {
    isPaused = !isPaused;
    const btn = document.getElementById('statusBtn');
    btn.innerText = isPaused ? "CONTINUAR" : "DETENER";
    btn.className = isPaused ? "stopped" : "running";

    if (isPaused) {
        if (websocket && websocket.readyState === WebSocket.OPEN) websocket.close();
        console.log("□ Pausado, WebSocket cerrado");
    } else {
        pendingCount = 0; pendingWriteIdx = 0; pendingReadIdx = 0;
        rollingCount = 0; rollingIdx = 0;
        fftCount = 0; fftIdx = 0;
        oscNeedsRender = true; fftNeedsRender = true;
        awaitingReady = false;
        _M = false;
        if (_T) { clearTimeout(_T); _T = null; }
        initWebSocket();
        console.log("▶ Reanudado, reconectando WebSocket");
    }
```

```
}

// --- Funciones del menú contextual ---
function applyChartSpeed(speedMmPerSec) {
    let newTbase = Math.round(REF_SAMPLES * (REF_SPEED / speedMmPerSec));
    newTbase = Math.min(10000, Math.max(50, newTbase));
    document.getElementById('tBase').value = newTbase;
    document.getElementById('tBaseNum').value = newTbase;
    updateOscSettings();
}

function applyAmplifier(ampMmPerUv) {
    let newVScale = REF_HEIGHT_MM / (ampMmPerUv * 1e6);
    newVScale = Math.min(7.0, Math.max(0.00005, newVScale));
    if (window.updateVScaleSlider) {
        window.updateVScaleSlider(newVScale);
    } else {
        document.getElementById('vScale').value = newVScale;
        document.getElementById('vScaleNum').value = newVScale.toExponential(2);
    }
    updateOscSettings();
}

function closeContextMenu() {
    contextMenu.style.display = 'none';
}

function setupContextMenu() {
    oscChartContainer.addEventListener('contextmenu', (e) => {
        if (currentTab !== 'time') return;
        e.preventDefault();

        contextMenu.style.left = e.clientX + 'px';
        contextMenu.style.top = e.clientY + 'px';
```

```
        contextMenu.style.display = 'block';

        setTimeout(() => {
            window.addEventListener('click', closeContextMenu, { once: true });
            window.addEventListener('contextmenu', closeContextMenu, { once: true });
        }, 10);
    });

    contextMenu.addEventListener('click', (e) => {
        const item = e.target.closest('.menu-item');
        if (!item) return;

        const action = item.dataset.action;
        if (action.startsWith('speed-')) {
            const speed = parseInt(action.split('-')[1]);
            applyChartSpeed(speed);
        } else if (action.startsWith('amp-')) {
            const amp = parseInt(action.split('-')[1]);
            applyAmplifier(amp);
        }
        closeContextMenu();
    });
}

window.onload = () => {
    initCharts();
    setupDualControl('filterFreq', 'filterFreqNum', recalcFilters);
    setupLogSlider('vScale', 'vScaleNum', updateOscSettings);
    setupDualControl('vOffset', 'vOffsetNum', updateOscSettings);
    setupDualControl('tBase', 'tBaseNum', updateOscSettings);
    setupDualControl('fftN', 'fftNNum', updateFFTSettings);
    setupDualControl('fftYMax', 'fftYMaxNum', updateFFTYAxis);
    setupDualControl('playbackSpeed', 'playbackSpeedNum', updatePlaybackSpeed);
    recalcFilters();
```

```
        updateOscSettings();
        updateFFTYAxis();
        initWebSocket();
        setupContextMenu();
        animationId = requestAnimationFrame(renderLoop);
    };

    window.addEventListener("resize", () => {
        const el = document.getElementById(currentTab === 'time' ? 'view-time' : 'view-freq');
        const dims = { width: el.clientWidth - 20, height: el.clientHeight - 20 };
        if(currentTab === 'time' && uplotOsc) uplotOsc.setSize(dims);
        else if(currentTab === 'freq' && uplotFFT) uplotFFT.setSize(dims);
    });
</script>
</body>
</html>”
```